



CYBER; CRA; Essential cybersecurity requirements for Virtualisation Execution Stack (VES) and Container Execution Stack (CES)

CAUTION:

Exercising one of the Principles of international standardization – Openness – and taking it to a different level, ETSI is piloting informal public consultations of the vertical standards in support of the Cyber Resilience Act at a much earlier stage than it is usual in the standardization world. In this context, please keep in mind that the standard draft herein is an INTERIM DRAFT, which expectably will be subject to substantial changes before its target publication date in the second semester of 2026.

Disclaimer

This **INTERIM DRAFT** document is provided for information and is for future development work within the ETSI Technical Committee CYBER EUSR Working Group (CYBER-EUSR) only. ETSI and its Members accept no liability for any further use/implementation of this Specification. Approved and published specifications and reports shall be obtained exclusively via the ETSI Documentation Service at <http://www.etsi.org/standards-search>

Commenting guidelines

Your comments to improve this early draft are very welcomed. To ensure the effectiveness of your contribution, please make sure to include the following elements in your comment:

1. Clear identification of the section of the draft your comment is referring to.
2. Objective proposal to delete content, add content or substitute content.
3. Concrete contribution (exact content you suggest including in the draft as an addition to, or in substitution of, existing content).
4. Brief rationale to justify the proposal (e.g. why the suggested content should be added an/or the existing content should be deleted or substituted). Please note that comments without concrete contributions will be noted but not acted upon by ETSI CYBER-EUSR. We trust and value your expertise and therefore expect you to take this opportunity to proactively contribute to solving the issues you find while analysing this early draft.

How to comment

To comment or provide feedback on the present interim draft please visit : <https://labs.etsi.org/rep/stan4cra/en-304-626> and submit your comments as “issues” following the guidelines provided on this site.

Alternatively, you may also send your comments to: cybersupport@etsi.org using the “[ETSI Commenting Format for Open Consultation.xlsx](https://docbox.etsi.org/CYBER/EUSR/OpenConsultation.xlsx)” available in <https://docbox.etsi.org/CYBER/EUSR/OpenConsultation.xlsx>



Reference

DEN/CYBER-EUS-0016

Keywords

CRA; Cybersecurity; Virtualisation Execution Stack (VES), Container Execution Stack (CES), Hypervisor, Container Runtime System (CRS)

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
[ETSI Search & Browse Standards](#) application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on [ETSI deliver](#) repository.

Users should be aware that the present document may be revised or have its status changed,
this information is available in the [Milestones listing](#).

If you find errors in the present document, please send your comments to
the relevant service listed under [Committee Support Staff](#).

If you find a security vulnerability in the present document, please report it through our
[Coordinated Vulnerability Disclosure \(CVD\)](#) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.

No recommendation as to products and services or vendors is made or should be implied.

In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm except as authorized by written permission of ETSI.

The content of the PDF version shall not be modified without the written authorization of ETSI.

The copyright and the foregoing restriction extend to reproduction in all media.

© ETSI 2025.
All rights reserved.

ETSI

Contents

Intellectual Property Rights	7
Foreword.....	7
Modal verbs terminology	7
Executive summary	8
Introduction	8
1 Scope	9
2 References	10
2.1 Normative references	10
2.2 Informative references	10
3 Definition of terms, symbols and abbreviations.....	11
3.1 Terms	11
3.2 Symbols	12
3.3 Abbreviations.....	12
4 Product architecture and components.....	13
4.1 VES.....	13
4.1.1 In-scope components.....	13
4.1.2 Security-relevant environmental dependencies (out of scope).....	14
4.2 CES.....	15
4.2.1 In-scope components.....	15
4.2.2 Security-relevant environmental dependencies (out of scope).....	15
5 Representative Use Cases.....	16
5.1 Purpose and Rationale	16
5.2 Use Cases for VESs	16
5.3 Use Cases for CES	17
5.4 Use Case to Risk Mapping Table.....	18
6 Threats.....	19
6.1 Overview	19
6.2 VES.....	20
6.2.1 Hyper Type I	20
6.2.2 Hyper Type II.....	21
6.2.3 Orchestration and Management system.....	22
6.3 CES.....	23
6.3.1 Container Orchestrator	23
6.3.2 Container Engine.....	24
6.3.3 Container Runtime System.....	25
7 Security Objectives	27
7.1 Isolation	27
7.2 Integrity Protection	27
7.3 Authentication.....	27
7.4 Authorisation	27
7.5 Confidentiality Protection.....	27
7.6 Availability and Resilience	27
7.7 Logging	27
7.8 Patches and Updates	27
7.9 Secure Configuration	27
7.10 Secure by Default	28
7.11 Data Minimisation	28
7.12 Time Synchronisation.....	28
8 Operational Environment Objectives	28
9 Risk-Based Requirement Classes/Categorisation and Application.....	29

9.1	Requirement Classes, Categorisation and Security Category Levels.....	29
9.1.1	Definition of Requirement Classes: Basic, Elevated, Advanced.....	29
9.1.2	Security Category Level Definitions.....	30
9.1.3	Requirement Categorisation.....	31
9.1.4	Guidance for Manufacturers.....	32
9.2	Requirement Application.....	32
9.2.1	SCL-Based Requirement Application.....	32
9.2.2	Requirement Application through Custom Security Profile Development.....	33
10	VES Security Requirements.....	33
10.1	Hypervisor Requirements.....	34
10.1.1	Isolation.....	34
10.1.1.1	VM Isolation.....	34
10.1.1.2	Control Plane Isolation.....	34
10.1.1.3	Network Plane Separation.....	34
10.1.2	Integrity Protection.....	35
10.1.2.1	Boot chain integrity verification.....	35
10.1.2.2	Guest VM image integrity verification.....	35
10.1.2.3	Runtime integrity protection.....	35
10.1.2.4	Remote attestation.....	35
10.1.3	Authentication.....	36
10.1.3.1	Administrative Authentication.....	36
10.1.3.2	Service Authentication.....	36
10.1.4	Authorisation.....	37
10.1.4.1	Administrative Authorisation.....	37
10.1.4.2	Service Authorisation.....	37
10.1.5	Confidentiality Protection.....	37
10.1.6	Availability and Resilience.....	38
10.1.7	Logging.....	38
10.1.8	Patches and Updates.....	38
10.1.9	Secure Configuration.....	39
10.1.10	Secure by Default.....	39
10.1.11	Data Minimization.....	39
10.1.12	Time Synchronisation.....	40
10.2	Applicability of Hypervisor Requirements to Type I and Type II.....	40
10.3	Management and Orchestration System Requirements.....	40
11	CES Security Requirements.....	41
11.1	CRS Requirements.....	41
11.1.1	Isolation.....	41
11.1.2	Integrity Protection.....	41
11.1.3	Authentication.....	42
11.1.4	Authorisation.....	42
11.1.5	Confidentiality Protection.....	42
11.1.6	Availability and Resilience.....	42
11.1.7	Logging.....	42
11.1.8	Patches and Updates.....	42
11.1.9	Secure Configuration.....	42
11.1.10	Secure by Default.....	42
11.1.11	Data Minimisation.....	42
11.1.12	Time Synchronisation.....	42
11.2	CE Requirements.....	42
11.3	CO Requirements.....	43
12	Assurance Security Requirements.....	44
12.1	Vulnerability Management for Open-Source Components.....	44
12.2	Software Bill of Materials (SBOM).....	44
13	Conformance Assessment.....	45
13.1	Assessment Methodology.....	45
13.2	Assessment for VM Isolation Requirements (A-VM-ISO).....	45
13.3	Assessment for Control Plane Isolation (A-CP-ISO).....	47
13.4	Assessment for Network Plane Separation (A-NP-ISO).....	48

13.5	Assessment for Boot Chain Integrity Verification (A-BT-VER).....	50
	Annex A (informative): CRA Mapping Table	52
	Annex B (informative): Illustrative Cybersecurity Risk Assessment Methodology	55
	B.1 Risk Assessment Process	55
	B.2 Risk Factor Scoring.....	56
	B.3 Calculation and Mapping to Likelihood/Impact Levels.....	58
	B.3.1 Likelihood Score	58
	B.3.2 Impact Score	58
	B.4 Risk Matrix	59
	B.5 Risk Methodology Applied to Use Cases.....	59
	B.5.1 Risk Evaluation of VES use cases.....	59
	Hypervisor Type I.....	59
	Hypervisor Type II	62
	B.5.2 Risk Evaluation of CES use cases.....	65
	CRS	65
	Annex C (normative): Threat-to-Requirement Traceability Matrix	68
	C.1 VES.....	68
	C.2 CES	69
	Annex D (informative): Illustrative implementation security controls	69
	D.1 VES.....	69
	D.1.1 Hypervisor.....	69
	D.1.1.1 Isolation.....	69
	D.1.1.1.1 VM Isolation	69
	D.1.1.1.2 Control Plane Isolation	70
	D.1.1.1.3 Network Plane Separation.....	70
	D.1.1.2 Integrity Protection	70
	D.1.1.2.1 Boot chain integrity.....	70
	D.1.1.2.2 Guest VM image integrity.....	70
	D.1.1.2.3 Runtime integrity protection	71
	D.1.1.2.4 Remote attestation.....	71
	D.1.1.3 Authentication	71
	D.1.1.3.1 Administrative Authentication	71
	D.1.1.3.2 Service Authentication	72
	D.1.1.4 Authorisation	72
	D.1.1.4.1 Administrative Authorisation.....	72
	D.1.1.4.2 Service Authorisation.....	72
	D.1.1.5 Confidentiality Protection.....	73
	D.1.1.6 Availability and Resilience.....	73
	D.1.1.7 Logging	73
	D.1.1.8 Patches and Updates	73
	D.1.1.9 Secure Configuration.....	74
	D.1.1.10 Secure by Default	74
	D.1.1.11 Data Minimisation	74
	D.1.1.12 Time Synchronisation.....	75
	D.1.2 Management and Orchestration System	75
	D.2 CES.....	75
	D.2.1 CRS	75
	D.2.1.1 Isolation	75
	D.2.1.2 Integrity Protection	75
	D.2.1.3 Authentication	75
	D.2.1.4 Authorisation	75
	D.2.1.5 Confidentiality Protection.....	75
	D.2.1.6 Availability and Resilience.....	75
	D.2.1.7 Logging	75
	D.2.1.8 Patches and Updates	75
	D.2.1.9 Secure Configuration.....	75
	D.2.1.10 Secure by Default	75
	D.2.1.11 Data Minimisation	75
	D.2.2 CE.....	75

D.2.3

CO

75

Annex (informative): Bibliography

76

Annex (informative): Change history

77

History

79

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs): Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the [ETSI IPR online database](#).

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM®** and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

ETSI Technical Committee Cybersecurity (TC CYBER) and is now submitted for the combined Public Enquiry and Vote phase of the ETSI EN Approval Procedure.

The present document has been prepared in response to the Commission's standardisation request C(2025)618 [i.3] to provide, in additions to its other uses, one voluntary means of conforming to the essential requirements of Regulation (EU) 2024/2847 [i.2] known as the Cyber Resilience Act (CRA). It corresponds to the new deliverable in line 35 of Annex I of the standardisation request.

Once the present document is cited in the Official Journal of the European Union under Regulation (EU) 2024/2847 [i.2], conformance with the normative clauses of the present document given in the table(s) in Annex A confers, to products with digital elements in the scope of the present document, a presumption of conformity with the corresponding essential requirements of that Regulation and associated EFTA regulations.

Proposed national transposition dates	
Date of latest announcement of this EN (doa):	3 months after ETSI publication
Date of latest publication of new National Standard or endorsement of this EN (dop/e):	6 months after doa
Date of withdrawal of any conflicting National Standard (dow):	18 months after doa

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the [ETSI Drafting Rules](#) (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

Executive summary

"Executive summary" clause should be deleted if not necessary.

Introduction

This European harmonised standard defines cybersecurity requirements applicable to Virtualisation Execution Stack (VES) and Container Execution Stack (CES). It applies to products with digital elements designed to host, manage, and isolate virtualised workloads and execution environments, using:

- Virtual machines (VMs): full system emulation with independent operating systems; or
- Containers: process isolation using OS-level mechanisms.

This standard supports compliance with Regulation (EU) 2024/2847, the Cyber Resilience Act, specifically addressing the essential cybersecurity requirements defined in Annex I, Parts I and II.

Commented [LP1]: This introduction paragraph is very good, but probably should be better placed in the Scope clause.

Commented [MH2R1]: After adding the definitions of hypervisor and CRS from the CRA in the scope, should we still keep them in the introduction? What should the 'Introduction' section normally contain in a harmonised standard?

1 Scope

This standard is based on the definitions provided in the Cyber Resilience Act (CRA) for hypervisors and container runtime systems as core components:

- Hypervisors are software products with digital elements that abstract and/or allocate computing resources and enable the execution and management of virtual machines that are logically separated from each other and/or from the physical hardware. Hypervisors may run directly on hardware (bare metal), on top of an operating system, or within another virtual machine (nested virtualisation). In the context of this category of products, a virtual machine is a software-defined logical separation of a computing environment, which includes a virtualised set of hardware resources (e.g., CPU, memory, storage, network interfaces) and typically hosts its own operating system. This category includes but is not limited to Type 1 hypervisors, Type 2 hypervisors, and hybrid hypervisors.
- Container runtime systems (CRS) are software products with digital elements that manage the execution and lifecycle of containers running on a single host operating system as isolated processes, allocating resources and allowing the management of individual containers. In the context of this category of products, a container is a software-based execution environment that encapsulates one or more software components and their dependencies in a single package, enabling it to run independently and consistently.

While the CRA definition identifies the core execution components, actual market products typically include additional elements beyond the hypervisor kernel or container runtime binary. These additional components provide essential management, orchestration, and operational capabilities that are necessary for real-world deployment and are therefore included within the scope of this standard.

This standard focuses on:

- **Virtualisation Execution Stack (VES):** set of software components responsible for deploying, running, and managing virtual machines (VMs) on shared physical infrastructure. It includes:
 - A hypervisor (Type I or Type II) that abstracts hardware and ensures isolation between VMs:
 - Type I hypervisors run directly on bare-metal hardware.
 - Type II hypervisors run on top of a host operating system.
 - An orchestration and management system that automates the deployment, scaling, and coordination of VMs across the infrastructure, and provides configuration, monitoring, and lifecycle control of the VMs.
- **Container Execution Stack (CES):** A layered set of software components enabling the deployment, management, and execution of containerised applications. It includes:
 - Container Runtime System (CRS): The Container Runtime System is the low-level software component responsible for creating, running, and managing containers on a host OS. It interfaces directly with the underlying system **kernel** to provide process isolation, resource control, and namespace management, and is responsible for executing container processes (e.g., using namespaces and control groups on Linux, or equivalent mechanisms on other operating systems).
 - Container Engine (CE): A Container Engine manages the full lifecycle of containers including image pulling, creation, execution, logging, and storage. It typically interacts with the container runtime system to perform low-level execution tasks and provides APIs or CLIs to manage containers.
 - Container Orchestrator (CO): A Container Orchestrator automates the deployment, scaling, networking, availability, and lifecycle management of containerized applications across clusters of machines. It coordinates how and where containers are scheduled to run and manages communication between them.

Accordingly, this standard defines security requirements not only for the core execution systems identified in the CRA but also for the broader product context in which these systems are deployed, ensuring alignment with market reality and comprehensive coverage of security risks.

Commented [LP3]: You are not reusing the technical description of the EU in your scope, copied below. Besides, the Hypervisor word is extensively used in the threat descriptions and requirement specifications, so it should be introduced at the beginning of this scope.

“Hypervisors and container runtime systems that support virtualised execution of operating systems and similar environments
-Hypervisors are software products with digital elements that abstract and/or allocate computing resources and enable the execution and management of virtual machines that are logically separated from each other and/or from the physical hardware. Hypervisors may run directly on hardware (bare metal), on top of an operating system, or within another virtual machine (nested virtualisation). In the context of this category of products, a virtual machine is a software-defined logical separation of a computing environment, which includes a virtualised set of hardware resources (e.g. CPU, memory, storage, network interfaces) and typically hosts its own operating system. This category includes but is not limited to type 1 hypervisors, type 2 hypervisors and hybrid hypervisors.

-Container runtime systems are software products with digital elements that manage the execution and lifecycle of containers running on a single host operating system as isolated processes, allocating resources and allowing the management of individual containers. In the context of this category of products, a container is a software-based execution environment that encapsulates one or more software components and their dependencies in a single package, enabling it to run independently and consistently.»

Commented [MH4R3]: Agreed. The definition has been added, along with an explanation of why the scope was expanded to cover additional components.

Commented [LP5]: This is a definition to be also included in clause 3.1, and acronyms in 3.2

Commented [MH6R5]: Added by Laure, idem for CES. Thanks

Commented [LP7]: Only Linux?
As a general rule, avoid using specific product names, standards should be technology neutral.

Commented [MH8R7]: Addressed

Commented [MH9R7]: This comment also applies to other parts of the standard and needs to be checked.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

Referenced documents which are not found to be publicly available in the expected location might be found in the [ETSI docbox](#).

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

- [1] CRA Regulation (EU) 2024/2847 - Cyber Resilience Act.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding but are not required for conformance to the present document.

- [i.1] Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act) (Text with EEA relevance)

[Regulation - 2024/2847 - EN - EUR-Lex](#)

- [i.2] Commission Implementing Regulation (EU) .../...of XXX on the technical description of the categories of important and critical products with digital elements pursuant to Regulation (EU) 2024/2847 of the European Parliament and of the Council (Text with EEA relevance)

- [i.3] C(2025)618 – Standardisation request M/606: Commission Implementing decision of 3.2.2025 on a standardisation request to the European Committee for Standardisation (CEN), the European Committee for Electrotechnical Standardisation (Cenelec) and the European Telecommunications Standards Institute (ETSI) as regards products with digital elements in support of Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act)

https://ec.europa.eu/growth/tools-databases/enorm/mandate/606_en

NIST SP 800-125A Rev. 1 – Security for Server Hypervisors

NIST SP 800-190 – Container Security Guide

CIS Benchmarks

NIAP PPs

Common Criteria PP_BASE_VIRT_V1.1

NIST SP 800-125A rev1 (Security Recommendations for Hypervisor Platforms)

NIST SP 800-53 rev5

NIST SP 800-207 (Zero Trust Architecture)

CIS VMware ESXi Benchmarks

MITRE ATT&CK for Virtualization Platforms

ETSI GS NFV-SEC 003 (NFV Security & Trust)

CCiTC Guidance for Cloud Virtualisation

NIAP Virtualization PP-Modules

[NIST SP 800-190] Application Container Security Guide

[MITRE ATT&CK] Container Techniques and Tactics

[CIS Benchmarks] Kubernetes, Docker, OpenShift

[PP_BASE_VIRTUALIZATION] Common Criteria Protection Profile (Containers Section)

[ETSI NFV-SEC] Virtualized Infrastructure Security

[CCiTC] Guidance for Cloud Evaluations

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the following terms definitions given in the CRA Regulation [i.1] apply:

<list of terms>.

In addition, the following terms apply:

Container Execution Stack (CES): A layered set of software components enabling the deployment, management, and execution of containerised applications

Hypervisor: A hypervisor is a software layer that creates and manages virtual machines (VMs) by abstracting and allocating physical hardware resources of a host system. It provides isolation between virtual machines and mediates access to the underlying CPU, memory, storage, and devices.

Virtualisation Execution Stack (VES): set of software components responsible for deploying, running, and managing virtual machines (VMs) on shared physical infrastructure

Virtual Machine (VM): It is a software-based abstraction of a physical computer system. It runs a guest operating system on top of a hypervisor, which abstracts the hardware resources, providing isolation between multiple VMs.

Tenant: A tenant is the fundamental, isolated unit of consumption, management, and billing within a virtualized or containerized environment. It represents a single, independent user or organization utilizing the shared infrastructure.

Multi-Tenancy:

- Multi-tenancy in a hypervisor environment means a single physical server hosts multiple, fully isolated Virtual Machines (VMs), each belonging to a different tenant.
- Multi-tenancy in a container environment means a single host machine or cluster runs multiple isolated containers and services, which may belong to different tenants.

Type I Hypervisor (Bare-Metal / Native): Operates directly on the host hardware, controlling hardware resources, scheduling virtual machines, and managing their isolation. The hypervisor itself forms the primary operating system for the virtualised environment (e.g., VMware ESXi, Microsoft Hyper-V, Xen, KVM where Linux kernel acts as the hypervisor).

Commented [LP10]: We will insert this at the beginning of the 3.1 clause of all verticals, not copying the definitions, only the list of terms.

Commented [LP11R10]: [L_202402847EN.000101_fm.xml](#)

Commented [MH12R10]: OK

Type II Hypervisor (Hosted): Operates as a software application running on top of a general-purpose host operating system (e.g., VMware Workstation, Oracle VirtualBox, Parallels Desktop). It relies on the host operating system for hardware access, scheduling, and many foundational system services.

Strong Authentication: Authentication that robustly verifies the identity of a user, service, or component to a level suitable for protecting administrative interfaces, control functions, or sensitive operations and data, where unauthorised access would pose a significant risk. This typically involves methods that are resistant to common attack vectors such as phishing, replay attacks, and credential stuffing.

Likelihood: The probability or frequency of a threat event occurring. This is influenced by various likelihood factors.

Impact: The magnitude of harm if a threat event materialises. This is determined by various impact factors.

Trusted Execution Environment (TEE): A Trusted Execution Environment is a secure area of a processor that ensures code and data loaded inside are protected with respect to confidentiality and integrity. A TEE provides an isolated execution environment that prevents unauthorized access or modification by the main operating system, hypervisor, or other applications. Examples include secure enclaves and confidential virtual machines.

Confidential Computing: It refers to the protection of data in use by performing computation in a hardware-based TEE. It allows processing of encrypted or sensitive data without exposing it to the rest of the system (e.g., OS, hypervisor, admin). This approach complements data-at-rest and data-in-transit protection with data-in-use protection.

MicroVM: A MicroVM is a lightweight virtual machine designed for minimal resource overhead, rapid startup, and high-density deployment. It typically offers a smaller attack surface and lower boot time compared to traditional VMs. MicroVMs are often used for serverless, container-like workloads, and confidential computing scenarios. They rely on minimal device emulation and limited kernel dependencies.

Container: A container is a lightweight, standalone, executable software package that includes everything needed to run a piece of software, including the code, runtime, system tools, libraries, and configuration. Containers isolate applications from the underlying host system and from each other, typically sharing the host OS kernel.

Commented [LP13]: Term also included in the CEN CENELEC vocabulary - check consistency across the board.

Commented [MH14R13]: Will be aligned following the assessment of terminology conflicts. This also applies to other terms that need to be harmonised.

3.2 Symbols

For the purposes of the present document, the [following] symbols [given in ... and the following] apply:

3.3 Abbreviations

[To be completed in a future version]

For the purposes of the present document, the [following] abbreviations [given in ... and the following] apply:

API: Application Programming Interface

CE: Container Engine

CES: Container Execution Stack

CLI: Command Line Interface

CO: Container Orchestrator

CRS: Container Runtime System

FaaS: Function as a Service

VES: Virtualized Execution Stack

VMM: Virtual Machine Monitor (Hypervisor Core)

VM: Virtual Machine

RBAC: Role-Based Access Control

MFA: Multi-Factor Authentication

SCL: Security Category Level

TLS: Transport Layer Security

TPM: Trusted Platform Module

SBOM: Software Bill of Materials

4 Product architecture and components

This standard distinguishes between product components (within the scope of this standard) and environmental dependencies (outside the scope of this standard but potentially security-relevant).

- In-scope components represent the functional building blocks that together define the product boundary as considered in this standard. The requirements specified in this document apply directly to these components.
- Environmental dependencies, by contrast, are not part of the product itself but belong to its operational environment (e.g., host operating system, firmware, or external services). These dependencies shall be secured in accordance with other standards (see clause 8).

In some cases, a product placed on the market may be delivered together with one or more out-of-scope components (for example, a VES bundled with a host OS). When this occurs, the manufacturer shall ensure that these bundled components also conform to the CRA, by applying the relevant harmonised standards where available.

4.1 VES

4.1.1 In-scope components

The following components are in the scope of the standard:

- Hypervisor core kernel or microkernel
- Hypervisor management interface or CLI/API (local or remote)
- Virtual Machine Monitor (VMM) and guest VM lifecycle controller
- Resource scheduling components (CPU, memory, I/O)
- Device virtualisation modules (e.g., disk controllers, SR-IOV support, paravirtualized device drivers, including any driver components running inside guest operating systems that directly interact with the host hypervisor)
- Management and Orchestration components
- Logging and auditing subsystems native to the hypervisor
- Security update enforcement mechanisms
- Virtual Trusted Platform Module (vTPM) functionality provided by the hypervisor for guest VMs.

The figure below highlights in blue the components that are within the scope of this standard. Components not highlighted in blue are considered out of scope.

VES - Type I Hypervisor

Commented [LP15]: It seems to me that what you are describing in this section is what you have illustrated in the visuals in the Scope Clause. I would move the diagrams from the Scope into this clause for consistency.

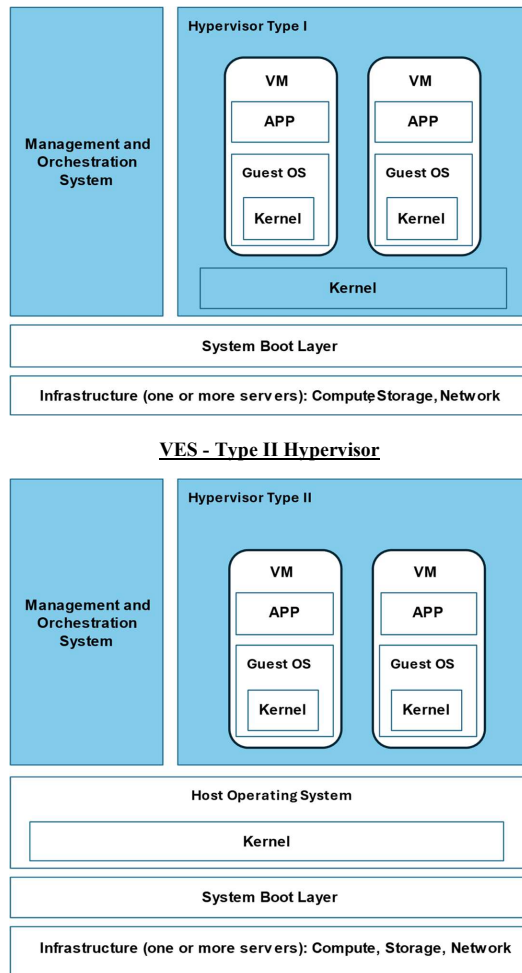
Commented [MH16R15]: OK, Done

Commented [LP17]: Are those independent components or components of the product we are considering in this standard?

Commented [MH18R17]: The listed items are not independent products but rather functional components that together form the boundary of the VES product as considered in this standard.

Commented [LP19]: Within the boundary => in the scope

Commented [MH20R19]: OK, done for VES and CES



4.1.2 Security-relevant environmental dependencies (out of scope)

The following are considered outside the scope of the standard but may be security-relevant environmental components:

- The Host Firmware and BIOS/UEFI
- The Host Operating System, in the case of Type 2 hypervisors
- Guest Operating Systems and virtualised application layers, with the exception of paravirtualized drivers that directly interface with the hypervisor.
- Virtual network infrastructure beyond the hypervisor switch (e.g., SDN controllers)
- External logging platforms (e.g., SIEMs)
- External attestation systems
- Hardware-level security controls (TPM, HSM) not embedded in the hypervisor or used to create a virtual equivalent (vTPM)
- Identity management systems, secrets vaults, and PKI

- Processes, tools, environments, or development/packaging formats whose primary purpose is to develop, build, or package guest operating system artifacts, including but not limited to OS installation media, virtual disk images, VM templates, build pipelines, compilers, and related image-building utilities.

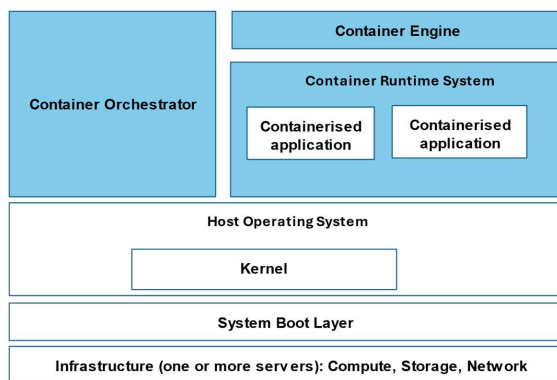
4.2 CES

4.2.1 In-scope components

The following components are in the scope of the standard:

- CRS: it is the lowest-level layer responsible for the execution and isolation of containers.
 - The core binary (e.g., runc, crun).
 - Enforcement modules that manage container isolation, resource control, and sandboxing through OS or application-level mechanisms (e.g., namespaces, microVMs, seccomp, cgroups).
 - The API and services that manage the runtime and its containers on a host (e.g., containerd, CRI-O).
- Container Engine: it is the mid-level layer that manage the container lifecycle.
 - The engine and its interfaces for building, pulling, pushing, and running container images (e.g., Docker, Podman, containerd).
 - Management of container images including the verification a signature at the host level.
 - Management of container networking at the host level (e.g., creation of network interfaces, bridge networks, and network namespace assignment).
 - Management of volumes for persistent or shared storage.
 - Management of plugins as supported by the engine.
- Container Orchestrator: it is the top-level layer that manages the entire lifecycle of applications and services across a cluster of machines (e.g., Kubernetes).
 - Features that ensure containers are deployed securely.
 - Image signature verification at the cluster level, secure scheduling to trusted nodes, and secrets management.
 - Policy enforcement that enforces security and isolation policies across the entire cluster (e.g. pod-level security policies and network isolation rules).
 - Lifecycle management that handles the scaling, networking, load balancing, and self-healing of containerised applications.

The figure below highlights in blue the components that are within the scope of this standard. Components not highlighted in blue are considered out of scope.



4.2.2 Security-relevant environmental dependencies (out of scope)

The following are considered outside the scope of the standard but may be security-relevant environmental components:

- The Host Operating System on which the CRS executes (OS kernel, drivers)

- Container registries (public/private)
- Container image build pipelines (CI/CD Pipelines)
- Third-party monitoring or security agents installed at runtime
- Identity and Access Management systems used for authentication/authorisation
- Underlying hardware platform (TPM, HSM, hardware root-of-trust)
- Processes, tools, environments, or development/packaging formats whose primary purpose is to develop, build, or package container artifacts, including but not limited to build pipelines, CI/CD systems, compilers, language-specific package managers, container image builders, image layers, and image manifests.

5 Representative Use Cases

5.1 Purpose and Rationale

In accordance with the proportionality principle mandated by the [CRA \[i.1\]](#), this clause defines representative use cases for VESs and CESs. These use cases illustrate typical deployment contexts that influence the applicability of security requirements.

Each use case is associated with a [risk level](#), derived directly from the application of the risk assessment methodology detailed in Annex B. While these use cases are representative, a more granular and detailed risk assessment will be necessary in scenarios where:

- The product combines characteristics of multiple use cases.
- The product does not align clearly with any single representative use case.
- The manufacturer identifies, through its own analysis, a threat exposure that is higher, lower, or different from that assumed in the representative use case.

To comply with CRA Annex I, Part I, (1), the product manufacturers shall apply a documented and effective risk assessment process to identify, evaluate, and address cybersecurity risks. The risk assessment methodology provided in Annex B is included for information purposes only and is not normative. The risk assessment methodology in Annex B is intended to illustrate one possible approach that may be adopted or replaced by an equivalent method already in use by the manufacturer.

5.2 Use Cases for VESs

The risk characteristics for each VES use case are identified based on the risk assessment methodology defined in Annex B.

UC-V1: VES for isolated, non-critical functions

- Description: A VES deployed on a dedicated host in an isolated environment. The host hosts a very small number of static VMs dedicated to non-critical functions. In this context, non-critical functions refer to operations whose compromise would not result in harm to human safety, disruption of critical infrastructure services, or exposure of sensitive data belonging to multiple users.
- Example environments:
 - Standalone servers or industrial PCs deployed in physically isolated networks,
 - Edge devices or gateways with highly restricted links,
 - Personal computers or laptops running a local hypervisor in a non-production environment.
- Illustrative business applications:
 - Isolated diagnostic data collection points in a factory,
 - Environmental monitoring systems for non-critical building zones,
 - Local telemetry units for non-essential parameters in a grid,
 - Read-only public information displays in industrial facilities (e.g., current temperature, non-operational metrics).
 - Consumer devices such as personal computers or laptops running local virtual machines for hobbyist testing, educational purposes, or personal experimentation.
- Risk level: Low (see Annex B.5.1)

Commented [LP21]: Use cases or Product context ? Terminology to be consistent across all vertical standards.

Commented [MH22R21]: 'Use Case' is used throughout the standard. This clause specifically defines the use cases. What does 'product context' mean?

Commented [LP23]: Always mention the reference index next to the references.

Commented [MH24R23]: OK

Commented [LP25]: Terminology «general risk level» and «security category level» to be consistent across the verticals.

Commented [MH26R25]: Agree. will check with other standards and align

Commented [LP27]: Are these only examples of environment or is this list exhaustive?

Commented [MH28R27]: only examples

- Main identified risks: These include malware exploiting unpatched systems in isolated industrial networks; unauthorized physical access to devices resulting in data corruption; and misconfigured hypervisors with default administrative credentials discovered in industrial telemetry units.

Commented [LP29]: Listing only risk «examples» doesn't seem appropriate in a cybersecurity standard. What about «main identified risks»?

Commented [MH30R29]: Yes agree, updated

UC-V2: VES for single-tenant cloud

- Description: A VES deployed within a controlled enterprise data centre environment, primarily supporting internal virtualised workloads and applications. These systems are part of an organisation's internal IT infrastructure.
- Example environments:
 - Enterprise private cloud platforms,
 - On-premises virtualisation clusters for internal workloads,
 - Internal development and test environments.
- Illustrative business applications:
 - Corporate IT service platforms,
 - Enterprise middleware supporting internal operations,
 - Internal microservices for business applications,
 - Private banking or insurance back-end systems.
- Risk level: Medium (see Annex B.5.2)
- Main identified risks: These include VM escapes exploiting vulnerabilities in enterprise virtualization platforms (e.g., VMware ESXi); privilege escalation attacks in hypervisor environments (e.g., KVM/QEMU) enabling host-level access; unauthorized internal access to sensitive data; compromised administrative credentials causing widespread system outages; and attacks leveraging known vulnerabilities in virtualization platforms to gain elevated privileges.

UC-V3: VES for multi-tenant cloud

- Description: A high-assurance VES forming the foundation of a public, private, or hybrid cloud platform, specifically designed to host untrusted tenant VMs from multiple distinct organizations or multiple independent units within the same organization. This scenario demands robust isolation and strong separation of concerns.
- Example environments:
 - Public IaaS clouds providing long-lived virtual machines and persistent resources,
 - Private clouds supporting multiple departments or projects with strong isolation needs,
 - FaaS for ephemeral, high-density workloads often based on microVMs, with extreme multi-tenancy and rapid scaling.
- Illustrative business applications:
 - Telecom core clouds hosting critical network functions.
 - Government virtual desktop deployments,
 - Financial services platforms hosted in public multi-tenant clouds,
 - eHealth applications requiring strict isolation of patient data.
- Risk level: High (see Annex B.5.3)
- Main identified risks: These include hypervisor (e.g., Xen/KVM) escape vulnerabilities exploited in multi-tenant environments; hardware-level side-channel data leaks (e.g., Meltdown, Spectre) affecting public cloud tenants; and denial-of-service attacks on multi-tenant hypervisors, impacting availability across customers (e.g., noisy neighbour exploits).

Commented [LP31]: What is KVM and Xen - Maybe you can include a reference here?

Commented [MH32R31]: Idem comment before, references could be added.

Commented [LP33]: Could we replace this by : hardware vulnerabilities (e.g. Meltdown and Spectre) ?

Commented [MH34R33]: Updated

5.3 Use Cases for CES

The risk characteristics for each CES use case are identified based on the risk assessment methodology defined in Annex B.

UC-C1: CES for isolated, non-critical functions

- Description: A CES deployed on a device with limited connectivity. This device typically performs non-critical functions, often in an isolated environment. These applications perform non-critical functions, meaning their compromise would not result in harm to human safety, disruption of critical infrastructure services, or exposure of sensitive data belonging to multiple users.
- Example environments:

- o Isolated test/dev environments,
 - o Basic controllers for non-critical functions.
 - o Consumer devices such as personal smartphones running isolated app containers for non-critical applications or laptops using local container engines (e.g., Docker Desktop, Podman) for personal projects.
- Illustrative business applications:
 - o Educational or hobbyist containerized workloads,
 - o Local telemetry or monitoring for non-critical parameters,
 - o Entertainment or personal productivity apps packaged in containers.
- Risk level: Low (see Annex B.5.4)
- Main identified risks: These include container escape vulnerabilities in container runtimes (e.g., runc); incidents of exposed container management daemons without TLS protection (e.g., Docker daemon) in isolated environments; and misconfigurations or unauthorized physical access leading to minor service disruption or data corruption.

UC-C2: CES for single-tenant cloud

- Description: A CES deployed within an enterprise's private data centre or on-premises infrastructure, serving internal enterprise applications.
- Example environments:
 - o Enterprise private cloud infrastructures,
 - o On-premises container platforms,
 - o Internal microservices architectures,
 - o Shared CES platforms hosting applications for multiple departments within the same enterprise.
- Illustrative business applications:
 - o Government services running on private clouds,
 - o Enterprise middleware platforms,
 - o Private banking application backends.
- Risk level: Medium (see Annex B.5.5)
- Main identified risks: These include misconfiguration incidents in container orchestration platforms (e.g., Kubernetes) exploited in enterprise environments; container breakouts through host mount privileges in development or test environments; privilege escalation attacks; unauthorized lateral movement across the network; and exploits targeting misconfigured container registries or insecure API endpoints.

UC-C3: CES for multi-tenant cloud

- Description: A CES executing dynamic, untrusted functions or microservices in a public, private, or hybrid cloud context, typically as part of a multi-tenant container platform. Multi-tenancy may involve workloads from different organizations or from independent units within the same organization.
- Example environments:
 - o Public container platforms supporting untrusted or third-party code execution,
 - o Private multi-tenant clouds hosting diverse workloads across departments or projects,
 - o Multi-tenant cloud infrastructures hosting diverse workloads from different organizations,
 - o Large-scale public cloud clusters offering ephemeral and high-density microservice deployments.
- Illustrative business applications:
 - o Government or telecom workloads requiring secure multi-tenancy,
 - o Finance and eID services built on public cloud container services,
 - o eHealth transformation workloads.
- Risk level: High (see Annex B.5.6)
- Main identified risks: These include API server exposures in container orchestration platforms (e.g., Kubernetes) enabling attackers to take control of clusters; container escape exploits abused in public cloud environments; supply chain attacks targeting public container images; and unauthorized data access between tenant workloads caused by misconfigured networking or weak isolation.

Commented [LP35]: I wouldn't consider the Finance and eID services as «environments», more like example of use cases/business applications.

Commented [MH36R35]: Done

Commented [LP37]: Same comments as above on eHealth.

Commented [MH38R37]: Done

Commented [LP39]: Could we avoid citing specific product names without changing the technical meaning?

Commented [MH40R39]: Done

Commented [LP41]: You have not defined the three Security Category Levels L1 2 3 in this document, that you then use in the clause 6 below. It seems to me that this clause 5.4 would be right place to define them. As - if my understanding is correct - they correspond to your Low/Medium/High risk level.

Commented [MH42R41]: At this stage, the outcome of the risk assessment is defined for each use case, with three possible risk levels: Low, Medium, and High. At this phase, no SCLs are assigned yet. The SCLs will be defined later, as sets of requirements relevant to each use case.

5.4 Use Case to Risk Mapping Table

Product	Use Case	Description	Risk Level	Annex Clause
---------	----------	-------------	------------	--------------

VES	UC-V1	VES for isolated, non-critical functions	Low	B.5.1
VES	UC-V2	VES for single-tenant cloud	Medium	B.5.2
VES	UC-V3	VES for multi-tenant cloud	High	B.5.3
CES	UC-C1	CES for isolated, non-critical functions	Low	B.5.4
CES	UC-C2	CES for single-tenant cloud	Medium	B.5.5
CES	UC-C3	CES for multi-tenant cloud	High	B.5.6

6 Threats

6.1 Overview

The following table maps identified cybersecurity threats to the specific components they affect within the scope of this standard: Hypervisor Type I, Hypervisor Type II, Orchestration & Management System, Container Orchestrator, Container Engine, and Container Runtime System.

The first column lists the threat categories considered in this standard, grouped by their primary security impact, such as interference or escape, integrity compromise, unauthorised access, data disclosure, denial of service, logging issues, malicious updates, insecure configurations, and supply chain compromise.

Each threat is referenced by its unique Threat ID, ensuring traceability to its detailed description in the threat catalogue in the following clauses. This mapping provides a clear overview of which threats are applicable to each component type, serving as the foundation for the risk assessment and for defining the corresponding security requirements and controls.

Threat	Hypervisor Type I	Hypervisor Type II	Orchestration & Management	Container Orchestrator	Container Engine	Container Runtime System
Interference / Escape	T-HYP1-INTF-ESC	T-HYP2-HOST-ABUSE	T-ORCH-CTRL-PLANE	T-CO-INTF-ESC	T-CE-INTF-ESC	T-CRS-INTF-ESC
Integrity Compromise	T-HYP1-INTEG-COMP	T-HYP2-INTEG-COMP	T-ORCH-INTEG-TRUST	T-CO-INTEG-TRUST	T-CE-INTEG-COMP	T-CRS-INTEG-COMP
Unauthorized Admin Access	T-HYP1-UNAUTH-ADMIN	T-HYP2-UNAUTH-ADMIN	T-ORCH-UNAUTH-ADMIN	T-CO-UNAUTH-ADMIN	T-CE-UNAUTH-ADMIN	T-CRS-UNAUTH-ADMIN
Unauthorized Privileged Actions	T-HYP1-UNAUTH-FUNC	T-HYP2-UNAUTH-FUNC	T-ORCH-UNAUTH-FUNC	T-CO-UNAUTH-FUNC	T-CE-UNAUTH-FUNC	T-CRS-UNAUTH-FUNC
Sensitive Data Disclosure	T-HYP1-DATA-DISC	T-HYP2-DATA-DISC	T-ORCH-DATA-DISC	T-CO-DATA-DISC	T-CE-DATA-DISC	T-CRS-DATA-DISC
DoS via Resource Exhaustion	T-HYP1-AVAIL-DOS	T-HYP2-AVAIL-DOS	T-ORCH-AVAIL-DOS	T-CO-AVAIL-DOS	T-CE-AVAIL-DOS	T-CRS-AVAIL-DOS
Insufficient / Tampered Logging	T-HYP1-LOG-INT	T-HYP2-LOG-INT	T-ORCH-LOG-INT	T-CO-LOG-INT	T-CE-LOG-INT	T-CRS-LOG-INT
Malicious / Compromised Update	T-HYP1-UPD-COMP	T-HYP2-UPD-COMP	T-ORCH-UPD-COMP	T-CO-UPD-COMP	T-CE-UPD-COMP	T-CRS-UPD-COMP

Insecure Configuration Changes	T-HYP1-CFG-COMP	T-HYP2-CFG-COMP	T-ORCH-CFG-COMP	T-CO-CFG-COMP	T-CE-CFG-COMP	T-CRS-CFG-COMP
Insecure Default Configuration	T-HYP1-DEF-COMP	T-HYP2-DEF-COMP	T-ORCH-DEF-COMP	T-CO-DEF-COMP	T-CE-DEF-COMP	T-CRS-DEF-COMP
Supply Chain Compromise	T-HYP1-SC-COMP	T-HYP2-SC-COMP	T-ORCH-SC-COMP	T-CO-SC-COMP	T-CE-SC-COMP	T-CRS-SC-COMP

6.2 VES

6.2.1 Hyper Type I

T-HYP1-INTF-ESC: Inter-VM interference and escape via hypervisor/host interfaces

A malicious VM exploits flaws in the hypervisor, device emulation, vSwitch, or shared hardware to read/alter another VM's memory/state or escape to the hypervisor/root.

T-HYP1-INTEG-COMP: Integrity compromise of workloads or platform components

A malicious actor tampers with hypervisor binaries, kernel modules, configuration files, or VM images on the host storage, injecting malicious code or disabling isolation/security controls. The compromised hypervisor continues to operate without detection by higher-level systems, allowing the attacker to manipulate or monitor workloads.

T-HYP1-UNAUTH-ADMIN: Unauthorized access to Type-1 hypervisor management interfaces

An attacker exploits missing or weak authentication on the hypervisor's direct management interfaces (e.g., vSphere Client, ESXi Shell, SSH, vendor-specific APIs). This may be due to default credentials, weak password policies, outdated authentication protocols, or disabled MFA.

T-HYP1-UNAUTH-FUNC: Unauthorized privileged actions or data access on a Type-1 hypervisor

An attacker with valid but limited credentials (e.g., VM operator role) or a compromised low-privileged API token exploits insufficient authorization enforcement on the hypervisor management plane to perform high-impact actions or access restricted data.

T-HYP1-DATA-DISC: Unauthorized disclosure of sensitive data in a Type-1 hypervisor environment

An attacker exploits vulnerabilities, weak encryption, or misconfigurations in the hypervisor or its management plane to gain unauthorized access to sensitive data from virtual machines or platform resources.

This may include data in use (e.g., reading contents directly from RAM or CPU registers of running workloads), data at rest (e.g., accessing virtual disk images or configuration files), and data in transit (e.g., intercepting network traffic between VMs).

T-HYP1-AVAIL-DOS: DoS via resource exhaustion in a Type-1 hypervisor environment

An attacker either from a malicious or compromised guest VM, or via external network/API access, consumes excessive CPU, memory, I/O bandwidth, or network capacity on the hypervisor host, leading to degraded performance or full unavailability of hosted VMs.

This also includes scenarios involving resource overcommitment, where CPU steal time, memory contention, or shared I/O bottlenecks impact the performance of co-located VMs, potentially leading to degraded availability even when individual VM resource limits are not exceeded.

T-HYP1-LOG-INT: Insufficient logging or log tampering in a Type-1 hypervisor environment

An attacker with access to a guest VM or hypervisor management interface exploits insufficient logging, disabled audit trails, or insecure log storage to hide malicious actions (e.g., VM configuration changes, hypervisor setting modifications). They may also tamper with existing logs to erase evidence of compromise.

T-HYP1-UPD-COMP: Malicious or compromised update in a Type-1 hypervisor environment

An attacker injects, replaces, or replays a hypervisor firmware or software update to gain control over the virtualization layer. Because Type-1 hypervisors run directly on hardware, compromise here provides full access to all guest VMs and host resources.

T-HYPI-CFG-COMP: Unauthorised or insecure configuration changes in a Type-1 hypervisor

An attacker gains direct or indirect access to hypervisor configuration interfaces (CLI, web UI, API) or underlying configuration files and modifies settings. Because Type-1 hypervisors control hardware-level virtualization, malicious changes here can weaken isolation, expose management services, or disable logging and security controls.

T-HYPI-DEF-COMP: Insecure default configuration in a Type-1 Hypervisor

The hypervisor ships with insecure default settings, such as enabled but unnecessary management services, default administrator credentials, disabled encryption, or open network ports on the management plane. If administrators deploy the hypervisor without hardening these defaults, attackers can exploit them to gain access.

T-HYPI-SC-COMP: Supply chain compromise in a Type-1 Hypervisor

An attacker compromises a trusted third-party component used in the hypervisor's build process, such as a low-level driver, cryptographic library, or firmware dependency, to inject a backdoor or exploitable vulnerability.

Because the malicious code is baked into the hypervisor's core, every VM running on the host inherits the exposure, giving the attacker a privileged foothold.

6.2.2 Hyper Type II

T-HYP2-HOST-ABUSE: Guest-to-host escalation via host OS dependency and shared services

A malicious VM abuses the host OS attack surface to access host resources or other VMs; weaker isolation than Type-1 increases risk.

T-HYP2-INTEG-COMP: Integrity compromise in hosted virtualization

An attacker compromises the host OS or hypervisor application binaries/configuration, or tampers with VM images in the host filesystem. Because Type-2 relies on the host OS, compromise of the host environment compromises the virtualization layer and workloads.

T-HYP2-UNAUTH-ADMIN: Unauthorized access to Type-2 hypervisor or host OS admin functions

An attacker exploits weak authentication on the hypervisor application or on the host OS's administrative accounts. Since Type-2 hypervisors run on a general-purpose OS, compromise of the host admin account grants full control over the hypervisor and all guest VMs.

T-HYP2-UNAUTH-FUNC: Unauthorized privileged actions or data access on a Type-2 hypervisor

An attacker with authenticated but restricted access to the hypervisor application or underlying OS exploits inadequate authorization to perform actions reserved for administrators. Since Type-2 hypervisors rely on the host OS for resource access, weak separation can also allow escalation to host-level privileges.

T-HYP2-DATA-DISC: Unauthorized disclosure of sensitive data in a Type-2 hypervisor environment

An attacker leverages weaknesses in the hosted hypervisor or host OS to read sensitive data belonging to VMs or the platform.

This may include data in use (e.g., extracting secrets from process memory), data at rest (e.g., accessing VM image files stored on the host), and data in transit (e.g., capturing inter-VM or host-VM communications).

T-HYP2-AVAIL-DOS: DoS via resource exhaustion in a Type-2 hypervisor environment

An attacker running on a guest VM or the host OS consumes excessive system resources (CPU, memory, storage, network), causing other VMs or the hypervisor process to stall or crash.

T-HYP2-LOG-INT: Insufficient logging or log tampering in a Type-2 hypervisor environment

An attacker running on the host OS or inside a VM exploits weak logging practices to erase or manipulate event records (e.g., VM lifecycle events, admin actions) to avoid detection. In a Type-2 setup, logs are often stored in user-accessible locations on the host, making them easier to modify.

T-HYP2-UPD-COMP: Malicious or compromised update in a Type-2 hypervisor environment

An attacker injects or manipulates updates to the hypervisor application running on top of the host OS. Because these updates often come via user-initiated downloads, attackers may deliver installers that have been modified with malicious code or downgrade the software to a vulnerable version.

T-HYP2-CFG-COMP: Unauthorised or insecure configuration changes in a Type-2 hypervisor

An attacker alters hypervisor configuration settings via the host OS environment or through the hypervisor's application interface. Because the hypervisor runs as a process on the host OS, it may inherit insecure host-level configurations or be modified by local malware.

T-HYP2-DEF-COMP: Insecure default configuration in a Type-2 Hypervisor

The hypervisor application is installed on a host OS with insecure defaults, such as permissive VM network modes, open host-guest shared folders, or guest control channels enabled without authentication. Users may deploy the product without disabling these features, exposing the host and other VMs to attacks.

T-HYP2-SC-COMP: Supply chain compromise in a Type-2 Hypervisor

A third-party library or driver used by the Type-2 hypervisor is compromised before it ships, either during the upstream project's development or in the packaging pipeline.

When the hypervisor is installed on a user's OS, the malicious dependency executes with host-level privileges, allowing the attacker to compromise both the hypervisor and the host OS.

6.2.3 Orchestration and Management system

T-ORCH-CTRL-PLANE: Control-plane abuse leading to cross-workload interference and privilege escalation

An attacker with (or who obtains) control-plane/API access manipulates scheduling, networking, storage or policies to read/impact other tenants' workloads or to break isolation at scale.

T-ORCH-INTEG-TRUST: Misplaced trust in compromised platform

An orchestrator and management system treats a hypervisor host as trustworthy without verifying its integrity (no attestation, or attestation ignored). A compromised host can feed falsified state/configuration data back to the orchestrator and run tampered workloads while appearing legitimate.

T-ORCH-UNAUTH-ADMIN: Unauthorized access to orchestration/management plane

An attacker gains unauthorized access to the orchestration platform's administrative APIs, dashboards, or CLI tools through weak authentication, missing identity federation controls, or compromised admin accounts. This provides cluster-wide control of all connected hypervisors and workloads.

T-ORCH-UNAUTH-FUNC: Unauthorized privileged actions or data access in orchestration/management systems

An attacker with authenticated but restricted access to the orchestration platform exploits insufficient authorization on APIs or dashboards to invoke privileged actions cluster-wide or access sensitive data.

T-ORCH-DATA-DISC: Unauthorized disclosure of sensitive data via virtualization orchestration/management system

An attacker abuses vulnerabilities, excessive privileges, or misconfigured access controls in the orchestration platform to obtain sensitive data from managed VMs or platform resources.

This may include data in use (e.g., retrieving memory dumps from running workloads), data at rest (e.g., accessing stored VM snapshots or configuration files), and data in transit (e.g., intercepting orchestrator-mediated traffic).

T-ORCH-AVAIL-DOS: DoS via resource exhaustion in virtualization orchestration/management systems

An attacker overwhelms the orchestration control plane, APIs, or schedulers with excessive requests, malformed workload definitions, or frequent provisioning/de-provisioning cycles. This can lead to unresponsiveness of management tools and inability to operate workloads.

T-ORCH-LOG-INT: Insufficient logging or log tampering in virtualization orchestration/management systems

An attacker targets the orchestration system's audit and logging functions to hide or falsify records of administrative actions, API calls, or workload lifecycle events. These systems often serve as the central source of truth, so tampering can mislead both human operators and automated security processes.

T-ORCH-UPD-COMP: Malicious or compromised update in virtualization orchestration/management systems

An attacker tampers with updates for the orchestration platform or its plugins/extensions. A compromised orchestration system can push malicious configurations or updates to all managed hypervisors and workloads, multiplying the impact.

T-ORCH-CFG-COMP: Unauthorised or insecure configuration changes in virtualization orchestration/management systems

An attacker changes cluster-wide or hypervisor-specific configurations through the orchestration platform's management interfaces or APIs. Because the orchestration system controls many hypervisors, this can propagate insecure settings to the entire environment.

T-ORCH-DEF-COMP: Insecure default configuration in virtualization orchestration/management systems

The orchestration system ships with insecure cluster-level defaults, such as default admin accounts, unencrypted management traffic, weak API tokens, or permissive role assignments. If these defaults are not changed before production use, attackers can quickly gain high-level control.

T-ORCH-SC-COMP: Supply chain compromise in virtualization orchestration/management systems

An attacker injects malicious code into a third-party module, API library, or plugin used in the orchestrator's control plane.

Because orchestrators have administrative access to all managed hosts, the compromise cascades to the entire infrastructure once deployed.

6.3 CES

6.3.1 Container Orchestrator

T-CO-INTF-ESC: Orchestrator-level cross-workload interference

An attacker with access to orchestration APIs or permissions in one namespace/project abuses orchestration misconfigurations, policy gaps, or network segmentation flaws to interfere with workloads managed by the orchestrator.

T-CO-INTEG-TRUST: Orchestrator trusts compromised nodes or components

The orchestrator schedules workloads onto a container host (node) or engine whose binaries, configuration, or runtime state have been tampered with, without verifying its integrity. This may be due to the absence of attestation, or attestation being ignored or spoofed.

T-CO-UNAUTH-ADMIN: Unauthorized access to orchestrator management plane

An attacker gains access to the orchestrator's administrative interfaces (web dashboards, CLI tools, APIs) due to weak or missing authentication controls, default credentials, or compromised admin accounts. Since the orchestrator manages scheduling, scaling, and policy enforcement for all workloads, compromise can result in complete cluster control.

T-CO-UNAUTH-FUNC: Unauthorized privileged operations or data access in the orchestrator

An attacker with valid but low-privileged credentials or a compromised service account exploits insufficient authorization enforcement in the orchestrator's API server, CLI tools, or dashboards to perform privileged actions or read restricted data.

T-CO-DATA-DISC: Unauthorized disclosure of sensitive data via the container orchestrator

An attacker exploits vulnerabilities, excessive privileges, or misconfigured access controls in the orchestrator's API server, dashboard, or CLI to obtain sensitive data from managed workloads or control-plane resources.

This may include data in use (e.g., reading container memory or runtime secrets), data at rest (e.g., accessing container images, volumes, or configuration files), and data in transit (e.g., capturing pod-to-pod or service communications).

T-CO-AVAIL-DOS: DoS via resource exhaustion in the container orchestrator

An attacker overwhelms the orchestrator's control plane, APIs, or schedulers with excessive requests, malformed deployment manifests, or frequent scaling operations. This can lead to delays or complete unavailability in workload scheduling, management, and monitoring.

T-CO-LOG-INT: Insufficient logging or log tampering in the Container Orchestrator

An attacker targets the orchestrator's event and audit logging systems to suppress or alter records of critical cluster events, administrative actions, or API calls. This can allow malicious changes—such as unauthorized deployments, RBAC policy modifications, or workload scheduling—to go undetected.

T-CO-UPD-COMP: Malicious or compromised update in the Container Orchestrator

An attacker injects, tampers with, or replays updates to the orchestrator's control plane components (API server, scheduler, controller manager) or its plugins/extensions.

A compromised orchestrator update could allow the attacker to modify workload scheduling, bypass RBAC rules, or deploy malicious workloads across all managed nodes.

T-CO-CFG-COMP: Unauthorised or insecure configuration changes in the Container Orchestrator

An attacker exploits insufficient protection of the orchestrator's management interfaces (API server, dashboard, CLI) or its configuration files (e.g., kube-apiserver flags, etcd store). Improperly secured or altered settings may weaken workload isolation, network segmentation, or RBAC enforcement.

T-CO-DEF-COMP: Insecure default configuration in the Container Orchestrator

The orchestrator ships with insecure defaults such as:

- Anonymous API access enabled.
- Default admin accounts without enforced password change.
- Unrestricted inter-pod communication (no default network policies).
- Insecure admission controller settings that allow unverified workloads.

If administrators deploy the orchestrator without hardening these defaults, attackers can exploit them to gain cluster-level control or pivot between workloads.

T-CO-SC-COMP: Supply chain compromise in a Container Orchestrator

An attacker injects malicious code into a third-party module, plugin, or API library used in the orchestrator's control plane.

Because the orchestrator controls all cluster workloads and nodes, the compromise propagates to the entire environment.

6.3.2 Container Engine

T-CE-INTF-ESC: Engine-level cross-container interference

A malicious container exploits weaknesses in engine-level isolation enforcement (configuration, API exposure, or plugin vulnerabilities) to interfere with other containers running under the same engine.

T-CE-INTEG-COMP: Integrity compromise of container engine and managed images

An attacker tampers with the container engine's binaries, configuration files, or locally cached container images. The compromised engine can then execute manipulated workloads or disable controls, while still appearing healthy to the orchestrator.

T-CE-UNAUTH-ADMIN: Unauthorized access to container engine control endpoints

An attacker gains access to the container engine's local or remote control interfaces (e.g., Docker API socket, containerd gRPC API) due to weak authentication, unprotected sockets, or lack of TLS. This gives direct control over container lifecycle operations and engine configuration.

T-CE-UNAUTH-FUNC: Unauthorized privileged operations or data access in the container engine

An attacker with authenticated but restricted engine API access (local socket or remote endpoint) exploits weak authorization to invoke high-risk operations beyond their intended role.

T-CE-DATA-DISC: Unauthorized disclosure of sensitive data via the container engine

An attacker gains unauthorized access to container image layers, configuration files, or secrets stored locally on the engine host due to weak encryption, insufficient file permissions, or misconfigured access to control sockets.

This may include data in use (e.g., reading secrets from running container processes), data at rest (e.g., accessing local container image storage), and data in transit (e.g., intercepting traffic between containers using the engine's default network).

T-CE-AVAIL-DOS: DoS via resource exhaustion in the container engine

An attacker exploits the container engine's local API or socket to trigger excessive image pulls, container starts/stops, or logging operations, consuming CPU, memory, disk I/O, or network bandwidth on the host.

T-CE-LOG-INT: Insufficient logging or log tampering in the Container Engine

An attacker with access to the container engine API or control socket manipulates lifecycle logs (start/stop events, image pulls, error messages) to remove or alter records of their actions. Since the engine directly manages container execution, these logs are often key for local forensics.

T-CE-UPD-COMP: Malicious or compromised update in the Container Engine

An attacker injects or tampers with updates to the container engine binaries or API services. Since the engine directly interfaces with the host OS and launches containers, a compromised update can be used to execute arbitrary code with elevated privileges.

T-CE-CFG-COMP: Unauthorised or insecure configuration changes in the Container Engine

An attacker alters container engine daemon configurations (e.g., Docker daemon JSON, container config) to weaken security defaults or allow unsafe container operations. Because the CE interacts with both the CRS and the host OS, malicious changes here can bypass isolation and security enforcement.

T-CE-DEF-COMP: Insecure default configuration in the Container Engine

The container engine ships with unsafe defaults, such as:

- Exposed Docker API socket without TLS or authentication.
- Default bridge network mode allowing unrestricted container-to-container traffic.
- Default ability to run privileged containers.

If these are left unchanged post-deployment, attackers can directly control containers or gain host-level access.

T-CE-SC-COMP: Supply chain compromise in a Container Engine

A trusted dependency used by the CE, such as a registry client library, image build tool, or network driver, is compromised during development or distribution.

This malicious code can be triggered when the CE pulls or builds images, or when it sets up container networking.

6.3.3 Container Runtime System

T-CRS-INTF-ESC: Runtime-level container breakout

A malicious container exploits flaws in OS-level isolation mechanisms or runtime implementation to escape its sandbox and interfere directly with the host or peer containers.

T-CRS-INTEG-COMP: Runtime-level tampering and unverified trust

An attacker modifies runtime binaries, configuration, or container state data (namespaces, cgroups, mounts) to disable or bypass security controls. Because the orchestrator and engine may not verify the runtime's state, a compromised runtime can operate undetected.

T-CRS-UNAUTH-ADMIN: Unauthorized access to container runtime control mechanisms

An attacker directly interacts with the container runtime binary or its control interfaces, bypassing higher-level engine or orchestrator authentication. This may occur if the runtime is invoked manually or through a compromised local account.

T-CRS-UNAUTH-FUNC: Unauthorized privileged operations or data access in the container runtime system

An attacker with authenticated local or delegated runtime access exploits insufficient authorization to manipulate running containers or modify runtime configurations directly.

T-CRS-DATA-DISC: Unauthorized disclosure of sensitive data via the container runtime system

An attacker exploits runtime-level weaknesses to read sensitive data from running containers or host resources that should be isolated.

This may include data in use (e.g., reading contents from container or host memory), data at rest (e.g., accessing mounted volumes or container filesystem layers), and data in transit (e.g., intercepting inter-container or container-host communications).

T-CRS-AVAIL-DOS: DoS via resource exhaustion in the container runtime system

An attacker running inside a container exploits runtime isolation weaknesses to monopolize CPU, memory, or I/O resources, or floods inter-container or container-to-host communication channels to degrade performance.

T-CRS-LOG-INT: Insufficient logging or log tampering in the Container Runtime System

An attacker operating inside a container exploits insufficient runtime logging or insecure log handling to conceal actions such as privilege escalation attempts, restricted syscalls, or file system access violations. The runtime's own log, if tampered with, may give a false picture of container activity.

T-CRS-UPD-COMP: Malicious or compromised update in the Container Runtime System

An attacker injects, tampers with, or replays updates to the low-level runtime responsible for creating and managing containers. Since the runtime enforces isolation (namespaces, cgroups, seccomp), a compromised update could remove or weaken these controls.

T-CRS-CFG-COMP: Unauthorised or insecure configuration changes in the Container Runtime System

An attacker modifies runtime-level configuration files or parameters that control isolation mechanisms (namespaces, seccomp profiles, cgroups). Changes here can directly weaken container isolation, resource limits, or syscall restrictions.

T-CRS-DEF-COMP: Insecure default configuration in the Container Runtime System

The runtime system ships with insecure isolation or resource control defaults, such as:

- No seccomp or AppArmor profile applied by default.
- No default cgroup limits for CPU, memory, or I/O.
- Filesystem mounts allowing container write access to sensitive host paths.

Without post-deployment hardening, attackers can abuse these defaults to escape isolation or perform denial-of-service attacks.

T-CRS-SC-COMP: Supply chain compromise in a Container Runtime System

An attacker compromises a low-level runtime dependency, such as a system call handling library, sandboxing module, or container image unpacker, used by the CRS.

The malicious code executes whenever the CRS starts or runs containers, potentially enabling container escape.

7 Security Objectives

This clause sets out the common security objectives for all product categories. These objectives provide the foundation and are later refined into specific requirements for VES and CES in the subsequent clauses.

7.1 Isolation

The product shall enforce isolation between workloads, administrative functions, and network domains to prevent unauthorised access, interference, or leakage across logical boundaries. Isolation shall be enforced by default, with configuration options available to adjust isolation levels in line with assessed risk and operational requirements.

7.2 Integrity Protection

The product shall ensure the integrity of platform components, configuration, and workloads by preventing unauthorised modification and by detecting tampering at rest and at runtime.

7.3 Authentication

The product shall ensure that all access to administrative interfaces, control functions, or sensitive operations is subject to strong authentication of users, services, or components.

7.4 Authorisation

The product shall enforce access control to ensure that only authorised users, services, or components can invoke functions or access data based on their assigned privileges.

7.5 Confidentiality Protection

The product shall ensure the confidentiality of sensitive data in transit, preventing unauthorised disclosure.

The product shall protect sensitive data at rest under its direct control, preventing its unauthorised disclosure.

The product shall support the use of host-level or external data-at-rest confidentiality controls for sensitive data stored on underlying operating systems or external storage accessed by the product.

The product shall support the use of host-level or external secure deletion mechanisms for sensitive data stored on underlying operating systems or external storage accessed by the product, to ensure the data cannot be recovered after its intended use is complete.

The product shall protect sensitive data in use under its direct control, preventing its unauthorised disclosure.

7.6 Availability and Resilience

The product shall ensure its own availability and resilience against denial-of-service attacks, resource exhaustion, and other attempts to degrade or disrupt its essential functions.

7.7 Logging

The product shall generate security-relevant logs, including access attempts, configuration changes, and workload actions, and ensure those logs are protected from tampering.

7.8 Patches and Updates

The product shall support secure update mechanisms, including cryptographic validation of update packages and protection against rollback. It shall also allow for reversion to a previous verified secure state.

7.9 Secure Configuration

The product shall provide mechanisms for secure configuration, allowing administrators to implement and maintain a secure operating posture, and shall protect these configurations from unauthorised modification.

7.10 Secure by Default

The product shall be delivered with secure default settings. Only services essential to its core functionality shall be enabled by default. Basic protection mechanisms shall be automatically active upon installation, requiring no additional action from the user.

7.11 Data Minimisation

The product shall ensure that only data strictly necessary for its secure and intended operation is collected, processed, and stored. This includes limiting logs, telemetry, and diagnostic information to essential elements, applying proportionate retention, and avoiding unnecessary exposure of sensitive information.

7.12 Time Synchronisation

The product shall ensure accurate and reliable time synchronisation for both its internal operations and security-related functions.

8 Operational Environment Objectives

Based on the out-of-scope lists provided for both VES and CES in clause 4, the Operational Environment (OE) components are those elements not part of the product and are outside the scope of this standard, but which shall function securely to support and preserve the security of the in-scope components (see clause 4).

The manufacturer's responsibility under this standard is limited to implementing the security requirements applicable to the in-scope components defined in clause 4.

OE components, as listed in this clause, are outside the scope of the manufacturer's implementation under this standard.

However, for the product to operate securely and as intended, these OE components should meet the security objectives stated in this clause.

Ensuring that the OE components fulfil these objectives is the responsibility of the product deployer, integrator, or operator.

Compliance of OE components with the CRA, for example by applying relevant harmonized standards or using other equivalent measures, is considered a precondition for secure operation.

OE component	Product	How it supports security	Illustrative security requirements
Boot manager	VES, CES	Provides the root of trust. Secure boot and firmware integrity checks prevent malicious code from running before the product starts.	EN 304 623 (CRA Boot Manager)
Host Operating System (for Type 2 hypervisors)	VES (Type 2 hypervisors), CES	Provides the execution environment; shall be hardened and patched to avoid compromise that could bypass product security.	EN 304 626 (CRA OS)
Guest Operating Systems and virtualised application layers	VES	Shall be patched and secured to prevent guest-to-host attacks or use of vulnerabilities in guest drivers to attack the VES.	EN 304 626 (CRA OS)
Virtual network infrastructure (e.g., SDN controllers)	VES, CES	Enforces network isolation and protects management/control traffic from interception or tampering.	EN 304 625 (CRA physical and virtual network interfaces)
External logging platforms (SIEMs)	VES, CES	Securely store, correlate, and alert on logs from the product for detection and response.	EN 304 622 (CRA SIEM)

Commented [MH43]: Should we retain this req. and is it relevant in the context of the CRA?

Commented [MH44R43]: this requirement is relevant in the context of CRA, but indirectly. CRA doesn't explicitly require time synchronisation. However, CRA does require manufacturers to ensure that the product's security functions operate correctly and that security logs are accurate, protected, and usable for incident detection and forensic analysis.

Commented [LP45]: Are those objectives reused later in the definition of the requirements ?

Commented [MH46R45]: No, this chapter serves only to identify the components that are part of the product's operational environment and support its security, but are not within the scope. This means they are not covered by the requirements. Some of these components are being addressed in other CRA standards currently under development. This chapter is not referenced later in the document.

External attestation systems	VES, CES	Verify the integrity of the product and its configuration state to remote verifiers.	TBC
Hardware-level security controls (TPM, HSM, TEE)	VES, CES	Store cryptographic keys, provide secure measurements, and enable trusted boot/attestation.	CEN (CRA security boxes)
Identity & Access Management systems, PKI	VES, CES	Authenticate and authorize administrators and services interacting with the product.	EN 304 624 (CRA PKI)
Image registries	VES, CES	Store and distribute container/VM images securely, with signature verification and access controls to prevent tampering.	TBC
Firewall	VES, CES	Enforce network segmentation and restrict exposure of management/control interfaces.	EN 304 636 (CRA firewalls, intrusion detection and prevention systems)
Storage systems	VES, CES	For use by Applications (containers/VMs). Support encryption at rest.	TBC
Hardware infrastructure (Processors, BMCs, NICs, storage controllers, etc.)	VES, CES	Provides the trusted foundation for VES and CES operation. Secure and up-to-date firmware ensures integrity of the boot chain, enforces hardware-level isolation (CPU, memory, I/O) and supports cryptographic protection (e.g., memory and storage encryption). Deployment shall follow vendor security recommendations, including timely firmware updates, network segregation of management interfaces (e.g., BMCs, NICs), and strict control of service exposure.	TBC

9 Risk-Based Requirement Classes/Categorisation and Application

9.1 Requirement Classes, Categorisation and Security Category Levels

9.1.1 Definition of Requirement Classes: Basic, Elevated, Advanced

To enable proportional application of security requirements under the CRA, requirements in this standard are expressed with requirement Classes: Basic, Elevated and Advanced. These levels indicate the depth or strength of the requirement.

Basic (B)

- Definition: The minimum class of requirement, providing fundamental protection.
- Characteristics:
 - Typically software-based or configuration safeguards.
 - Establishes a baseline protection expected in all products.
 - Suitable for low-risk environments.

Elevated (E)

- Definition: Intermediate class of protection, addressing increased exposure.

Commented [LP47]: To be aligned across all verticals, I would suggest changing «Standard» to «Medium», see below.

Commented [MH48R47]: 'Elevated' is used as 'Medium' is already one of the risk level

- Characteristics:
 - Builds upon or replaces Basic with stronger mechanisms.
 - Leverages hardware-assisted features or stronger cryptographic protections.
 - Suitable for medium-risk environments.

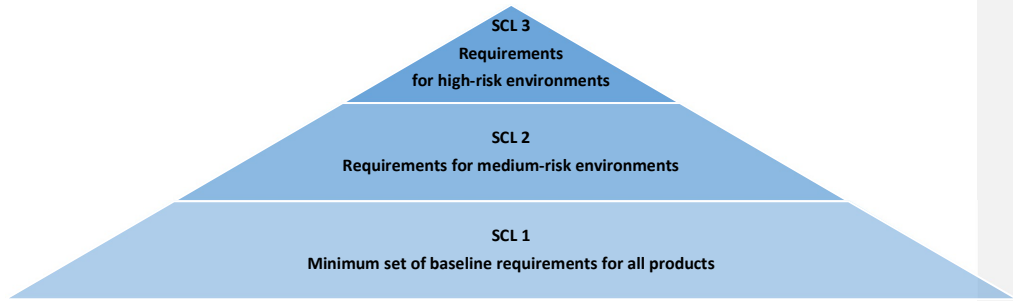
Advanced (A)

- Definition: The highest class of protection, incorporating the strongest available safeguards.
- Characteristics:
 - Provides resilience against advanced threats.
 - Incorporates hardware roots of trust and runtime protections.
 - Suitable for high-risk environments.

9.1.2 Security Category Level Definitions

To support risk-proportional application of security requirements under the CRA, the standard defines three Security Category Levels (SCL) for security requirements. The defined SCLs support manufacturers in applying a risk-based approach, helping to determine applicable requirements aligned with the intended use case and proportionality principles under the CRA.

Level	Description
SCL 1	A minimum set of baseline cybersecurity requirements applicable to all products. These requirements are essential for fundamental security posture, particularly for use cases where compromise results in low risk (e.g., UC-V1, UC-C1). This level corresponds to products assessed as Low Risk according to the methodology in Annex B.
SCL 2	Requirements for medium-risk environments. Depending on the requirement categories defined in clause 9.1.3: (i) cumulative requirements include all Basic + Elevated elements; (ii) exclusive requirements apply at the Elevated level only; (iii) single-class requirements at Elevated are tied to this SCL. This corresponds to products assessed as Medium Risk in Annex B.
SCL 3	Requirements for high-risk environments. Depending on the requirement categories defined in clause 9.1.3: (i) cumulative requirements include Basic + Elevated + Advanced; (ii) exclusive requirements apply at the Advanced level only; (iii) single-class requirements at Advanced are tied to this SCL. This corresponds to products assessed as High Risk in Annex B.



Commented [LP49]: Here you are coming back to environments and use cases, it is rather confusing.

Commented [MH50R49]: Replaced

Commented [MH51]: To be updated. The selection of requirements is not only cumulative; it can also be exclusive or stand-alone.

Commented [LP52R51]: You are defining the SCL here, whereas you have already used the three levels in the previous clauses. You should reconsider the order of the clauses to avoid this for someone reading the standard in its sequence.

Commented [MH53R51]: order changed

9.1.3 Requirement Categorisation

The requirements in this document are defined using three categories: Cumulative, Exclusive, and Single-Class. Each category specifies how a requirement applies across the different classes (Basic, Standard, Advanced). The following subclauses describe these categories, providing guidance to implementers on how to apply them.

(1) Cumulative Requirements (Additive)

- Definition: Higher classes include all lower classes. Each class adds more rigor on top of the previous one.
- Rationale: This model applies when protections build in layers and depend on the foundation provided by lower classes.
- Application: If a manufacturer is assigned SCL 3 (Advanced), then all requirements at Basic, Elevated, and Advanced shall be implemented.
- Example – VM Isolation:
 - Basic: CPU and memory isolation using software-based controls.
 - Elevated: Basic + hardware-assisted virtualization and IOMMU separation.
 - Advanced: Elevated + mitigations for side-channel attacks.

(2) Exclusive Requirements (Non-Additive)

- Definition: Only one class applies. Classes represent alternative approaches, not cumulative layers.
- Rationale: This model applies when requirements define mutually exclusive mechanisms, where implementing one makes the others unnecessary.
- Application: The manufacturer implements only the requirement at the class corresponding to the product's assigned SCL.
- Example – Image Integrity Verification:
 - Basic: Hash verification of VM/container images before execution.
 - Elevated: Signature verification of images against a trusted public key.
 - Advanced: Signature verification with keys stored in a hardware root of trust.

(3) Single-Class Requirements

- Definition: A requirement is defined at only one class (Basic, Elevated, or Advanced). No variations exist at the other classes.
- Rationale: Some controls cannot be meaningfully scaled across classes and are best expressed at a single strength level.
- Application: A Single-Class requirement is tied to the SCL corresponding to its class:
 - Basic: SCL 1
 - Elevated: SCL 2
 - Advanced: SCL 3

The manufacturer implements the requirement only when the product is assigned to the SCL that matches the class.

- Example:
 - Advanced-only (SCL3): Remote attestation of the hypervisor's runtime state.

Commented [LP54]: Remember that implementers need to have a clear specification of how to comply with the CRA. May I suggest the following rephrasing (if I understood you well):
«The requirements in the present document are defined in three different ways, either in a Cumulative, Exclusive and/or Single-Label way. The following provides explanation on the requirement categorisation.»

Commented [MH55R54]: Agree, updated

9.1.4 Guidance for Manufacturers

To identify applicable requirements:

1. Determine product use case and risk level (see clause 5 and Annex B).
2. Identify requirement categorisation:
 - If cumulative, implement all lower classes up to the assigned SCL.
 - If exclusive, implement only the class that matches the assigned SCL.
 - If single-class, implement the one requirement defined at that class.

Commented [LP56]: I would not use the word «when selecting», rather «to identify». Would you agree?

Commented [MH57R56]: OK, Done

9.2 Requirement Application

This standard defines two approaches to identify the applicable security requirements. Both are grounded in risk assessment and aligned with the CRA [i.1] proportionality principle.

- The SCL-based approach shall be applied where the product aligns with one of the representative use cases defined in clause 5.
- The Custom Security Profile Development approach shall be used when representative use cases or Annex B risk levels do not apply, with requirements determined and justified through a documented risk analysis.

Commented [LP58]: While I understand the «complementarity» of the two approaches, I am unsure that allowing choice of approach to the manufacturers is suitable for a Harmonised Standard. We will need to validate this during the HAS assessment. You have to use wording that is as prescriptive as possible.

Commented [MH59R58]: Updated

9.2.1 SCL-Based Requirement Application

This approach provides manufacturers with predefined use cases mapped to Security Category Levels (SCLs) (see clause 9.1.2). Each SCL bundles the appropriate requirements for the assessed risk level (Low, Medium, High).

The representative use cases defined in clause 5 are reference cases to support risk-based alignment of requirements. They do not exhaustively cover all deployment scenarios.

Manufacturers may define alternative or product-specific use cases, provided that:

- A risk assessment is conducted (see guidance in Annex B) to clearly document the specific risk characteristics, threat exposure, and potential impacts of the product's environment.
- The assigned SCL is justified based on the outcome of the risk assessment, considering the product's exposure, complexity, and intended purpose.
- Documentation justifying the risk assessment and assigned SCL is maintained for conformity assessment purposes.

Step 1 – Determine product use case and associated risk level

Manufacturers shall begin by identifying the use case for their product (see reference cases in clause 5). The risk level for these use cases is pre-determined based on the application of the methodology in Annex B:

- If the product aligns with UC-V1 or UC-C1 → corresponds to a Low-risk use case.
- If the product aligns with UC-V2 or UC-C2 → corresponds to a Medium-risk use case.
- If the product aligns with UC-V3 or UC-C3 → corresponds to a High-risk use case.

Manufacturers may also define their own use case. In such instances, the cybersecurity risk assessment methodology from Annex B could be applied to determine the product's specific risk level (Low, Medium, or High), with the risk characteristics and threat exposure fully documented.

Step 2 – Map use case and determined risk level to SCL and apply requirements

Use Case	Risk Level	SCL 1	SCL 2	SCL 3	Applicable requirements
UC-V1 / UC-C1	Low	x			Implement all requirements marked Basic, including cumulative, exclusive, and single-class Basic.

Commented [LP60]: Could we change the title of this column, for clarity, from «Notes» to «Applicable requirements»?

Commented [MH61R60]: OK, Done

UC-V2 / UC-C2	Medium		x		Implement all requirements marked Basic and Elevated, including cumulative and exclusive Elevated, and single-class Elevated.
UC-V3 / UC-C3	High			x	Implement all requirements marked Basic, Elevated, and Advanced, including cumulative and exclusive Advanced, and single-class Advanced.

9.2.2 Requirement Application through Custom Security Profile Development

Manufacturers shall apply the Custom Security Profile Development approach when their product use case does not clearly align with any representative use case, when the deployment context results in different risk levels than those assumed in Annex B, or when the manufacturer applies its own documented risk assessment methodology. In this approach, the application of requirements shall be fully justified through a documented risk analysis.

This approach requires manufacturers to derive a tailored security profile directly from the threats and risks identified for the product. Annex C of this standard provides a normative threat-to-requirement mapping, with requirements categorized as Basic (B), Elevated (E), and Advanced (A), and further classified as Cumulative, Exclusive, or Single-Class. Manufacturers shall apply this mapping when tailoring requirements to their specific product risk assessment.

Step 1 - Assessment of an individual risk level for each identified threat

- The manufacturer shall identify all applicable threats to the product using the threat catalogue in clause 6.
- For each identified threat, the manufacturer shall determine likelihood and impact to calculate the associated risk level, either by applying the methodology in Annex B or by using an equivalent documented risk methodology.
- The outcome of this step shall be a complete threat list, each associated with a risk rating.

Step 2 - Application of requirements based on the assessed risk level of each individual threat

- For each identified threat, the manufacturer shall determine the corresponding requirement(s) from the normative mapping in Annex C.
- The categorisation rules shall apply:
 - Cumulative: all requirements up to the level corresponding to the assessed risk (e.g., Basic + Elevated for Medium risk).
 - Exclusive: only the requirement at the level corresponding to the assessed risk (e.g., Elevated requirement for Medium risk, without inheriting Basic).
 - Single-Class: the requirement applies only at the class where it is defined (e.g., an Elevated-only requirement applies exclusively for Medium risk).

Step 3 - Documentation and conformity evidence

- The manufacturer shall document, for each identified threat:
 - a) the assessed risk level,
 - b) the applicable requirement(s) from Annex C, including their class and categorisation, and
 - c) a justification that the applicable requirement(s) reduce the risk to an acceptable level.
- The resulting custom security profile and supporting documentation shall form part of the manufacturer's technical file and shall be made available for conformity assessment under the CRA.

This process shall produce a custom security profile that is fully risk-based, tailored to the product's context, and compliant with the normative threat-to-requirement mapping defined in Annex C.

10 VES Security Requirements

This clause defines specific security requirements for VESs, instantiating the security objectives outlined in Clause 7.

10.1 Hypervisor Requirements

10.1.1 Isolation

10.1.1.1 VM Isolation

Classes are cumulative

Basic

REQ-H-VM-ISO-001: The Hypervisor shall enforce logical separation between virtual machines at CPU, memory, and I/O levels using built-in process and memory space isolation.

Elevated

REQ-H-VM-ISO-002: The Hypervisor shall implement strong isolation mechanisms at CPU, memory, and I/O levels, , with the capability to leverage hardware-assisted isolation features when available on the underlying platform.

Advanced

REQ-H-VM-ISO-003: The Hypervisor shall strengthen isolation with mitigations against known side-channel attacks to ensure robust separation between virtual machines, with the capability to leverage hardware-assisted side-channel protections provided by the underlying platform.

10.1.1.2 Control Plane Isolation

Classes are cumulative

Basic

REQ-H-CP-ISO-001: The Hypervisor shall isolate its administrative functions and interfaces from guest virtual machines through internal access control.

Elevated

REQ-H-CP-ISO-002: The Hypervisor shall enforce dedicated logical separation between guest traffic and administrative interfaces.

Advanced

REQ-H-CP-ISO-003: The Hypervisor shall enforce physical or cryptographic separation for administrative interfaces to prevent lateral movement.

10.1.1.3 Network Plane Separation

Classes are cumulative

Basic

REQ-H-NP-ISO-001: The Hypervisor shall logically segregate management, guest, and host network segments using virtual network separation mechanisms.

Elevated

REQ-H-NP-ISO-002: The Hypervisor shall enforce ACLs and filtering rules to prevent cross-plane traffic.

Advanced

REQ-H-NP-ISO-003: The Hypervisor shall enforce physical network separation or cryptographic isolation for management traffic.

10.1.2 Integrity Protection

10.1.2.1 Boot chain integrity verification

Classes are exclusive

Basic

REQ-H-B-INT-001: The Hypervisor shall verify the integrity of its kernel or microkernel before exposing any guest workload interfaces or services, by computing a cryptographic hash and comparing it against a trusted reference value.

Elevated

REQ-H-B-INT-002: The Hypervisor shall verify the integrity of its bootloader and kernel/microkernel before exposing any guest workload interfaces or services, using a chained trust mechanism based on digital signatures.

Advanced

REQ-H-B-INT-003: The Hypervisor shall verify the integrity of its entire boot chain, including firmware, bootloader, kernel/microkernel, and management services, using digital signatures, with the capability to leverage a hardware-based root of trust provided by the underlying platform to validate each stage in the chain of trust.

10.1.2.2 Guest VM image integrity verification

Classes are exclusive

Basic

REQ-H-IMG-INT-001: The Hypervisor shall verify the integrity of guest VM disk images before execution or provisioning, by computing a cryptographic hash and comparing it against a trusted reference value.

Elevated

REQ-H-IMG-INT-002: The Hypervisor shall verify both the integrity and authenticity of guest VM disk images before execution or provisioning, using digital signatures validated against a trusted public key or certificate.

Advanced

REQ-H-IMG-INT-003: The Hypervisor shall verify the integrity and authenticity of guest VM disk images before execution or provisioning, using digital signatures with the capability to leverage a hardware-based root of trust for storing and validating the trust anchor.

10.1.2.3 Runtime integrity protection

Classes are exclusive

Elevated

REQ-H-RP-INT-001: The Hypervisor shall prevent, detect, and reject unauthorized or integrity-compromising runtime modifications to its security-critical configuration state, device drivers, or privileged memory regions using software-based runtime integrity protections.

Advanced

REQ-H-RP-INT-002: The Hypervisor shall prevent, detect, and reject unauthorized or integrity-compromising runtime modifications to its security-critical configuration state, device drivers, and privileged memory regions with the capability to leverage hardware-assisted runtime integrity protections provided by the underlying platform.

10.1.2.4 Remote attestation

Single Class

Advanced

REQ-H-RA-INT-001: The Hypervisor shall support the capability to perform measured boot and attestation of its bootloader, kernel/microkernel, critical configuration, and essential runtime state to an authorised verifier by leveraging hardware-based roots of trust provided by the underlying platform.

10.1.3 Authentication

Classes are exclusive

10.1.3.1 Administrative Authentication

Basic

REQ-H-ADMIN-AUTH-001: The hypervisor shall require authentication with a unique, non-default password meeting defined complexity requirements for access to all administrative interfaces, including local CLI, remote APIs, and VM management consoles.

Elevated

REQ-H-ADMIN-AUTH-002: The hypervisor shall support cryptographic key-based authentication for administrative access and allow administrators to disable password-based login where possible.

- Algorithms and keys shall meet current cryptographic best practices (cryptographic profiles will be aligned with ENISA EUCC ACM and ETSI ESI TS 119 312).
- Private keys shall be protected from unauthorized extraction or reuse.

Advanced

REQ-H-ADMIN-AUTH-003: The hypervisor shall support certificate-based authentication across all administrative interfaces, including CLI, APIs, and management consoles.

- Certificates shall be validated against a trusted Public Key Infrastructure (PKI) and verified for validity, expiration, and revocation status.
- The hypervisor shall support the capability to leverage hardware security modules or trusted platform services to securely store and protect cryptographic keys and credentials, where such platform capabilities exist.

10.1.3.2 Service Authentication

Basic

REQ-H-SERV-AUTH-001: The hypervisor shall require all external services, including orchestration systems, monitoring tools, and backup agents, to authenticate before being granted access to management functions or data.

- Each service shall use unique, non-default credentials.
- Shared or hard-coded credentials shall not be permitted.
- The hypervisor shall implement an account lockout mechanism for service accounts after a configurable number of failed login attempts.

Elevated

REQ-H-SERV-AUTH-002: The hypervisor shall support certificate-based mutual authentication for all services accessing management functions or data.

- Algorithms and keys shall meet current cryptographic best practices (cryptographic profiles will be aligned with ENISA EUCC ACM and ETSI ESI TS 119 312).
- Private keys shall be protected from unauthorized extraction or reuse.

Advanced

REQ-H-SERV-AUTH-003: The hypervisor shall support certificate-based mutual authentication for all service interactions, validated against a trusted PKI.

- Certificates shall be bound to specific service identities.
- The hypervisor shall support the capability to leverage hardware-protected storage for private keys and service credentials.

10.1.4 Authorisation

Classes are exclusive

10.1.4.1 Administrative Authorisation

Basic

REQ-H-ADMIN-AUTHZ-001: The hypervisor shall restrict all administrative actions to authenticated accounts and enforce a default-deny policy for access to management functions.

Elevated

REQ-H-ADMIN-AUTHZ-002: The hypervisor shall enforce granular, role-based access control (RBAC) or attribute-based access control (ABAC) for administrative accounts, ensuring that accounts are granted only the minimum privileges required for their role.

Advanced

REQ-H-ADMIN-AUTHZ-003: The hypervisor shall enforce fine-grained authorization policies for administrative accounts, including separation of duties and support for time-bound or just-in-time privilege elevation, with the capability to integrate with external identity and policy decision systems where available.

10.1.4.2 Service Authorisation

Basic

REQ-H-SERV-AUTHZ-001: The hypervisor shall enforce a default-deny policy for all service accounts, ensuring that service accounts are only permitted to perform actions explicitly authorize.

Elevated

REQ-H-SERV-AUTHZ-002: The hypervisor shall enforce granular authorization for service accounts, binding allowed actions to their authenticated identity. Each service account shall be configured with least-privilege permissions.

Advanced

REQ-H-SERV-AUTHZ-003: The hypervisor shall enforce fine-grained, policy-driven authorization for service accounts, binding permissions to cryptographic service identities, supporting context-aware enforcement, with the capability to integrate with external authorization systems where available.

10.1.5 Confidentiality Protection

Classes are cumulative

Basic

REQ-H-CONF-001: The Hypervisor shall support encrypted storage for sensitive data at rest, including but not limited to VM images, snapshot files, configuration files, and sensitive log data.

Elevated

REQ-H-CONF-002: The Hypervisor shall support encrypted communications for management interfaces and inter-VM traffic and VM migration traffic where applicable.

Advanced

REQ-H-CONF-003: The Type 1 hypervisor shall possess the capability to orchestrate and provision guest virtual machines with hardware-backed Trusted Execution Environments.

REQ-H-CONF-004: The Type 2 hypervisor shall possess the capability to leverage hardware-backed Trusted Execution Environments via the underlying host operating system, to provision and manage guest virtual machines with TEE protection.

Commented [MH62]: It has been decided to include these requirements for review by the team.

10.1.6 Availability and Resilience

Classes are cumulativeBasic

The Hypervisor shall detect and recover from common operational faults without requiring a reboot, to maintain availability of running guest VMs.

Elevated

The Hypervisor shall support high-availability clustering or failover mechanisms to resume guest VM operations within the recovery time defined by the administrator.

Advanced

The Hypervisor shall provide continuous state synchronization across clustered instances to enable seamless failover with no more data loss than allowed by the defined recovery point objective (RPO).

10.1.7 Logging

Classes are cumulativeBasic

The Hypervisor shall generate audit logs for all administrative actions and security events, including authentication attempts, configuration changes, and VM lifecycle operations.

Elevated

The Hypervisor shall protect audit logs from unauthorized access or modification, and provide secure export to an external log management or SIEM system.

Advanced

The Hypervisor shall support cryptographic signing and time-stamping of audit logs to ensure non-repudiation and integrity verification during forensic analysis.

10.1.8 Patches and Updates

Classes are cumulativeBasic

The Hypervisor shall provide a mechanism to apply security patches without requiring a full reinstallation.

Elevated

The Hypervisor shall verify the authenticity and integrity of all updates before installation, using digital signatures validated against a trusted key.

Advanced

The Hypervisor shall support live patching of critical components to remediate vulnerabilities without interrupting running workloads.

10.1.9 Secure Configuration

Classes are cumulativeBasic

The Hypervisor shall provide administrators with the ability to configure only the features and services necessary for its intended operation.

Elevated

The Hypervisor shall enforce configuration validation to prevent insecure parameter values from being applied.

Advanced

The Hypervisor shall support configuration baselines that can be automatically validated against a vendor-provided or administrator-defined secure configuration profile.

10.1.10 Secure by Default

Classes are cumulativeBasic

The Hypervisor shall be shipped with default settings that minimize the attack surface, including disabled unused services, strong default access controls, and secure default networking rules.

Elevated

The Hypervisor shall require administrators to explicitly enable high-risk features and shall provide warnings about their security implications.

Advanced

The Hypervisor shall apply adaptive hardening measures during deployment based on detected environment and workload sensitivity, enabling relevant protections automatically.

10.1.11 Data Minimization

Classes are cumulativeBasic

The Hypervisor shall limit log content, telemetry, and diagnostic data to information required for operation, troubleshooting, and security monitoring. Sensitive data shall not be stored or transmitted in logs or telemetry.

NOTE: Sensitive data may include plaintext passwords, full cryptographic keys, or complete VM memory contents.

Elevated

The Hypervisor shall provide configuration options to restrict the categories of operational data collected by disabling collection of specific log types or metrics.

Advanced

The Hypervisor shall automatically apply data minimization rules during log generation and diagnostic export, including:

- Redaction of sensitive values from logs.
- Truncation or masking of non-essential payloads in diagnostic files.
- Automatic deletion of temporary diagnostic data after export.

NOTE: Sensitive values may include API tokens, secret keys, and other credentials.

10.1.12 Time Synchronisation

Basic

REQ-H-TIME-001: The hypervisor shall be configured to synchronize its clock with a trusted time source to maintain accurate time for all normal operations and logging.

REQ-H-TIME-002: The hypervisor shall provide a mechanism for all guest virtual machines (VMs) to synchronize their time with the hypervisor's clock.

Elevated

REQ-H-TIME-003: The hypervisor shall synchronize with at least two independent, trusted time sources to ensure redundancy and reliability.

- All security-relevant events, including authentication attempts, configuration changes, and VM lifecycle actions, shall be time-stamped using the synchronized clock.
- If synchronization fails or the clock drifts beyond a defined threshold, the hypervisor shall raise a security alert and log the event.

Advanced

REQ-H-TIME-004: The hypervisor shall implement cryptographically authenticated time synchronization to prevent tampering with time sources.

10.2 Applicability of Hypervisor Requirements to Type I and Type II

To support proportional implementation of security requirements across different hypervisor architectures, the following table provides a mapping of each requirement to its applicability for Type I and Type II hypervisors:

[To be completed in a future version]

Requirement ID	Type I	Type II	Notes
	x		

10.3 Management and Orchestration System Requirements

[To be completed in a future version]

Authentication & Access Control

The M&O system shall require multi-factor authentication for all administrative access, whether via local console or web interface.

The M&O system shall enforce RBAC, ensuring each account is assigned only the minimum privileges necessary for its role.

The M&O system shall authenticate and authorize all API access using a token-based mechanism.

The M&O system shall support rate limiting on API requests to prevent abuse and denial-of-service attacks.

Secure Configuration

The M&O system shall be delivered with secure default settings, including least-privilege access roles, disabled unused services or ports, and enforced secure communication protocols.

The M&O system shall require explicit authorization before applying any configuration changes to the hypervisor or managed workloads.

The M&O system shall require explicit authorization before applying any changes to its own configuration, including but not limited to user roles and permissions, network settings, orchestration policies and update mechanisms.

Communication Security

The M&O system shall encrypt all management and orchestration communications, including those between distributed M&O components, between the M&O system and managed hypervisors, and between the M&O system and managed workloads. Encryption shall use industry-standard protocols with certificate-based mutual authentication where applicable.

EXAMPLE: TLS 1.2 or 1.3

Integrity Security

The M&O system shall, at import and again immediately before execution, verify the integrity and authenticity of orchestration artifacts (templates, playbooks, and scripts) using digital signatures validated against a trusted keyring; alternatively, for immutable vendor artifacts, verification may use a reference cryptographic checksum from a trusted catalog. Artifacts that fail verification shall not be stored or executed; the event shall be logged and administrators shall be alerted. The M&O system shall allow authorized administrators to register and revoke trusted signing keys.

The M&O system shall validate the integrity and authenticity of all third-party plugins, extensions, or integration modules before activation.

Logging & Monitoring

The M&O system shall log all administrative actions, including user logins, configuration changes, VM lifecycle events (create, delete, migrate), orchestration workflows, and security-related alerts.

The M&O system shall protect stored logs against tampering and support secure export to external logging or SIEM systems.

Patch Management

The M&O system shall provide a mechanism to securely obtain, verify (via digital signatures), and apply updates to itself and the managed hypervisors.

11 CES Security Requirements

This clause defines specific security requirements for CESs, instantiating the security objectives outlined in Clause 7.

11.1 CRS Requirements

11.1.1 Isolation

REQ-C-ISO-001: The CRS shall enforce strong logical separation between containers at the process, filesystem, network, and resource usage levels using OS-level isolation mechanisms.

REQ-C-ISO-002: The CRS shall enforce separation between container workloads and host administrative functions, including the runtime's own control interfaces.

REQ-C-ISO-003: The CRS shall ensure its control interface is logically or physically isolated from the container network data plane to prevent unauthorized access from workloads.

11.1.2 Integrity Protection

REQ-C-INTEG-002: The CRS shall prevent unauthorised modifications to a container's runtime configuration.

REQ-C-INTEG-003: The CRS shall protect its own components and its management interfaces and internal control logic from being modified, bypassed, or misused by container workloads, preventing API misuse or privilege escalation.

REQ-C-INTEG-004: The CRS shall provide integrity evidence of the instantiated container and its runtime policies to an authorized attestation service.

11.1.3 Authentication

REQ-C-AUTH-001: The CRS shall enforce strong authentication for any access to its control interfaces, including CLIs and APIs, from any external component or agent, before accepting requests to start, stop, or modify containers or their configurations.

11.1.4 Authorisation

REQ-C-AC-001: The CRS shall enforce access control to prevent unauthorized execution, entry, or modification of containers and their resource assignments. It shall reject control actions issued by unauthenticated or unauthorized entities.

REQ-C-AC-002: The CRS shall enforce policy-based restrictions on access to host-level resources, including devices, system calls, and filesystem paths.

11.1.5 Confidentiality Protection

REQ-C-CONF-001: The CRS shall support native or pluggable mechanisms to encrypt container images and sensitive data within containers at rest, including secrets, configuration files, and application data. It shall support enforcement of encryption policies and secure integration with external key management systems.

REQ-C-CONF-002: The CRS shall ensure the confidentiality of all communications with container registries, enforcing the use of cryptographic protocols for image pulls and metadata transfers.

REQ-C-CONF-003: The CRS shall provide support the execution of containers within hardware-backed TEEs, ensuring the confidentiality and integrity of the container's memory and state.

Commented [MH63]: It has been decided to include this requirement for review by the team.

11.1.6 Availability and Resilience

[To be defined in a future version]

11.1.7 Logging

[To be defined in a future version]

11.1.8 Patches and Updates

[To be defined in a future version]

11.1.9 Secure Configuration

[To be defined in a future version]

11.1.10 Secure by Default

[To be defined in a future version]

11.1.11 Data Minimisation

[To be defined in a future version]

11.1.12 Time Synchronisation

[To be defined in a future version]

11.2 CE Requirements

Isolation

The CE shall provide a secure interface to configure and invoke CRS operations for the creation, execution, and termination of containers. It shall ensure that the underlying CRS enforces strong logical separation consistently across all containers, as required.

The CE shall prevent container workloads from accessing host administrative functions by enforcing non-privileged execution, restricting host resource access, and disabling insecure defaults, including privileged containers and host mounts.

The CE shall ensure its administrative interface is logically or physically isolated from container network traffic and is accessible only to authorized host processes.

Integrity protection

REQ-C-INTEG-001: The CE shall verify the integrity and authenticity of container images and associated metadata, including manifests, before deployment or execution. This shall include validation of image digests and digital signatures using trusted keys or authorities. Images that fail verification shall not be made available to the container runtime for execution.

The CE shall restrict interfaces and commands that allow post-instantiation modification of container runtime parameters, unless performed as part of a container restart.

The CE shall prevent containers from accessing its administrative interfaces, including control APIs and UNIX sockets, by enforcing strict file permissions and namespace separation. It shall not expose its interfaces to container workloads by default.

Authentication

The CE shall enforce authentication on all interfaces used by orchestration components or external agents to manage containers. It shall ensure that only authenticated actors are allowed to execute workload lifecycle or configuration operations.

Authorisation

The CE shall enforce access control on all operations that affect container lifecycle or resource usage. It shall evaluate each request based on the authenticated identity of the caller and their assigned privileges, using a role-based or attribute-based policy model.

The CE shall support configuration of workload-level security policies that restrict access to sensitive host resources, including devices, filesystem paths, and syscalls. The CE shall translate these policies into runtime enforcement parameters for the CRS.

Confidentiality

The CE shall provide configuration options to enable encryption at rest for container images and volumes and pass required metadata to the CRS.

The CE shall ensure that its administrative control interfaces and all communications with external services are protected using encrypted connections, preventing the unauthorized disclosure of sensitive commands or data in transit.

11.3 CO Requirements

Isolation

The CO shall ensure its management and orchestration interfaces are logically or physically isolated from the container data plane.

Integrity protection

The CO shall provide mechanisms to enforce immutability of container specifications once a workload has been deployed, including admission policies and resource mutation restrictions.

The CO shall prevent deployment of container workloads that can access or modify the control plane, by enforcing admission policies, RBAC, and default-deny security postures for privileged features.

Authentication

Authorisation

The CO shall enforce fine-grained access policies on workload and resource control operations through Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC), ensuring that only authorized users or components can manage container deployments and configurations.

Confidentiality

The CO shall ensure the confidentiality of all internal cluster communication, including control plane traffic and communication between workloads, by enforcing encryption.

The CO shall provide support for scheduling and managing containers into TEE-backed environments, including the capability to apply security policies to such workloads.

12 Assurance Security Requirements

[To be defined in a future version]

Examples: Vulnerability management, supply chain (SBOM), Security testing, etc.

NOTE: This set of requirements relates to assurance aspects, not to functional capabilities that must be supported by the product itself. Rather, they concern processes within the manufacturer, such as secure development, lifecycle management, testing and vulnerability handling.

Inputs from CRA horizontal standards (e.g., PT01) can be reused and referenced in this clause, with appropriate refinements to reflect the specific context of VESs and CESSs.

12.1 Vulnerability Management for Open-Source Components

Open-source components form a significant part of VES and CES implementations. Effective use of community-driven security alerts and patches is essential to reduce exposure time. Regression testing ensures that rapid security fixes do not introduce unintended disruptions in operational environments.

REQ-ASSUR-VULM-001: For products incorporating open-source software (OSS), the manufacturer shall establish processes to leverage OSS community vulnerability reporting and resolution mechanisms. This includes timely monitoring of upstream advisories and security patches. The manufacturer shall ensure that applied updates are regression tested, covering both Long-Term Support (LTS) releases and relevant interim point releases, to maintain product stability and security.

12.2 Software Bill of Materials (SBOM)

REQ-ASSUR-SBOM-001: The manufacturer shall provide a Software Bill of Materials (SBOM) for VES and CES products (see in-scope components of the VES and CES as defined in clauses 4.1.1 and 4.2.1), documenting all software components, dependencies, and their versions.

REQ-ASSUR-SBOM-002: For VES, the SBOM shall include (at minimum):

- Hypervisor kernel or microkernel,
- Hypervisor management interfaces (CLI/API),
- Virtual Machine Monitor (VMM) and lifecycle controllers,
- Resource scheduling components (CPU, memory, I/O),
- Device virtualization modules.
- Hardware-related dependencies bundled with the VES product.

REQ-ASSUR-SBOM-003: For CES, the SBOM shall include (at minimum):

- CRS: runtime binaries, enforcement modules, APIs/services.
- Container Engine: engine binaries, image management tools, container networking modules (bridge, namespace managers), storage/volume plugins, supported extensions.

- Container Orchestrator: orchestrator binaries, admission controllers, scheduling modules, secrets management, network policy enforcement, cluster lifecycle management components.
- Hardware-related dependencies bundled with the CES product.

REQ-ASSUR-SBOM-004: The SBOM shall:

- Be maintained across the lifecycle of the VES/CES product, updated with each patch, release, or update.
- Be provided in a standardized machine-readable format to support automated tooling.
- Clearly identify open-source dependencies and third-party libraries included in the distribution.

NOTE: In this clause examples are provided to illustrate typical components and formats that may fall under the SBOM requirements. These examples are non-exhaustive and non-normative. For instance:

- Device virtualization modules may include SR-IOV drivers, paravirtualized device drivers, or host-guest interface modules.
- CRS runtime binaries may include runc or crun.
- CRS enforcement modules may include namespaces, seccomp, cgroups, microVMs.
- CRS APIs/services may include containerd, CRI-O.
- Container engine binaries may include Docker or Podman.
- Container orchestrator binaries may include Kubernetes.
- Machine-readable SBOM formats may include SPDX or CycloneDX.
- Hardware-related dependencies may include vTPM firmware, drivers for SR-IOV NICs, storage controllers, or other hardware-specific modules.

13 Conformance Assessment

13.1 Assessment Methodology

This clause defines the methodology for assessing conformity with the VES and CES security requirements specified in this standard.

Each requirement is associated with an assessment description that follows a structured format:

- **Assessment Reference:** Identifies the link to the exact requirement ID(s).
- **Assessment Objective:** Defines the security property or capability that shall be verified, ensuring that the assessment remains focused on the intent of the requirement.
- **Assessment Preparation:** Describes the environment, setup, and preconditions required before executing the test.
- **Assessment Activities:** Provides execution steps to be performed. Activities are designed to cover the necessary rigor depending on whether the requirement is Basic, Elevated, or Advanced.
- **Assignment of Verdict:** Defines the pass/fail criteria. The assessment is considered successful if the requirement's protection goals are demonstrably met; it fails if unauthorized access, modification, or bypass is possible, or if required security capabilities are unsupported.
- **Supporting Evidence:** Lists the artefacts to be collected and documented, such as logs, configuration files, screenshots, vendor documentation, and test results. Evidence ensures traceability and allows independent review.

13.2 Assessment for VM Isolation Requirements (A-VM-ISO)

Assessment Reference

- REQ-H-VM-ISO-001 (Basic)
- REQ-H-VM-ISO-002 (Elevated)
- REQ-H-VM-ISO-003 (Advanced)

Commented [MH64]: The proposition of test case has been presented as simplified PT2 structure. o confirmation yet about the acceptance of that approach however there is no objection, and the commission is in favor of simplification. Some experts recommended to have a look on the TS 103 701 "Conformance Assessment of Baseline Requirements" for consumer IoT devices, it specifies a conformance assessment methodology for consumer IoT devices, for addressing the mandatory and recommended provisions as well as conditions and complements of ETSI TS 103 645 / ETSI EN 303 645 "Baseline Requirements" for IOT by defining test cases and assessment criteria for each provision

Commented [LP65R64]: At this stage Mohamad I would not allow anymore options.

If you could keep a one to one correspondance :
One requirement = one way to do conformance assessment providing a non disputable and repeatable output regardless of who performs the assesment, that would be great!

Assessment Objective

Verify that the hypervisor enforces logical and hardware-assisted isolation between VMs, and that advanced deployments include demonstrable mitigations against known side-channel attacks, ensuring robust VM separation based on the chosen rigor level.

Assessment Preparation

REQ-H-VM-ISO-001 (Basic): Deploy a test environment consisting of at least two non-trusted Guest VMs and one host running the Hypervisor under test. Ensure full access to the hypervisor management interface.

REQ-H-VM-ISO-002 (Elevated): All items required for Basic, and: ensure the capability to enable/disable and configure all hardware-assisted isolation features (e.g., IOMMU, specific CPU security extensions).

REQ-H-VM-ISO-003 (Advanced): All items required for Elevated, and: The Manufacturer provides documentation detailing the supported CPU architectures and the implementation details of all side-channel mitigations. The Evaluator prepares penetration testing tools targeting known side-channel attacks for Step 3.

Assessment Activities (Cumulative Steps)

Step 1 – Basic (REQ-H-VM-ISO-001)

1. Start two guest VMs.
2. Attempt direct access from one VM to:
 - o Memory of the other VM.
 - o CPU registers belonging to another VM.
 - o I/O devices assigned to another VM.
3. Verify that all access attempts are rejected or isolated.

Expected Result: Cross-VM access is blocked. Hypervisor enforces logical separation using its built-in memory and process isolation.

Step 2 – Elevated (REQ-H-VM-ISO-002)

1. Enable all available hardware-assisted virtualization features for isolation.
2. Configure one VM with passthrough hardware.
3. Attempt to bypass isolation by injecting unauthorized I/O or DMA requests from the passthrough VM.
4. Review hypervisor logs/configuration to confirm hardware-assisted isolation is actively configured.

Expected Result: Unauthorized I/O/DMA requests are rejected or properly mediated. Hypervisor demonstrates the capability to leverage hardware-assisted features for stronger I/O and memory isolation.

Step 3 – Advanced (REQ-H-VM-ISO-003)

1. Select the representative hardware platform from the supported CPU architectures.
2. Confirm through configuration review and vendor documentation that side-channel mitigations are implemented and enabled.
3. Conduct a targeted VM escape penetration test using a known side-channel attack (e.g., cache timing or speculative execution principles) from within a guest VM, attempting to infer data or affect the host/another VM on the representative platform.
4. Attempt to infer data or establish a communication channel with the Host or a co-resident VM using the side-channel attack vectors.

Expected Result: The hypervisor's implemented mitigations successfully suppress or nullify the side-channel attack, preventing the inference of data or any measurable communication across isolation boundaries on the tested representative hardware.

Assignment of Verdict

The final verdict is assigned based on the highest level successfully verified during the assessment. The assessment activities are cumulative.

- PASS for Basic: Requires Step 1 to pass.
- PASS for Elevated: Requires Step 1 AND Step 2 to pass.
- PASS for Advanced: Requires Step 1 AND Step 2 AND Step 3 to pass.
- FAIL Condition: The overall assessment for a target level is FAIL if any required step for that level (or any preceding step) fails.

Supporting Evidence

REQ-H-VM-ISO-001 (Basic): Test logs showing blocked cross-VM functional access attempts (Memory, CPU, I/O).

REQ-H-VM-ISO-002 (Elevated): All evidence required for Basic, and: Hypervisor configuration and feature enablement logs proving the use of hardware-assisted isolation features. Test logs showing blocked I/O or DMA requests.

REQ-H-VM-ISO-003 (Advanced): All evidence required for Elevated, and: Vendor documentation confirming side-channel mitigations are implemented and enabled. Manufacturer's architectural statement (defining the CPU families/mitigation dependencies). Test logs detailing the failed side-channel penetration test attempts.

NOTE: The representative platform is the single hardware configuration selected by the Evaluator for the active penetration test (Step 3). This platform is chosen from the supported CPU architectures.

13.3 Assessment for Control Plane Isolation (A-CP-ISO)

Assessment Reference

- REQ-H-CP-ISO-001 (Basic)
- REQ-H-CP-ISO-002 (Elevated)
- REQ-H-CP-ISO-003 (Advanced)

Assessment Objective

Verify that the hypervisor effectively isolates its administrative control plane (management functions and interfaces) from potential interference, access, or lateral movement originating from guest VMs, scaled according to the rigor level.

Assessment Preparation

REQ-H-CP-ISO-001 (Basic): Configure a test environment with at least one Guest VM capable of network communication and one hypervisor management terminal. Ensure access to the hypervisor's internal Access Control List (ACL) configuration.

REQ-H-CP-ISO-002 (Elevated): All items required for Basic, and: Ensure access to the hypervisor's virtual network settings (VLANs, virtual switches) and host firewall configuration.

REQ-H-CP-ISO-003 (Advanced): All items required for Elevated, and: Obtain documentation detailing the cryptographic implementation used for securing administrative communication channels (if cryptographic separation is used).

Assessment Activities

Step 1 – Basic (REQ-H-CP-ISO-001)

1. From within a privileged user session on the Guest VM, attempt to discover, scan, and connect to the Hypervisor's administrative interfaces (e.g., management web interface, API endpoint, SSH daemon running on the host).

2. Attempt to interact with the hypervisor's internal access control functions via hypercalls or APIs that manage other VMs or host resources.
3. Audit the hypervisor's internal access control policy to confirm the explicit denial of VM-originated access to host-critical functions.

Expected Result: All attempts to access or interact with administrative functions are rejected by the hypervisor's internal access control layer.

Step 2 – Elevated (REQ-H-CP-ISO-002)

1. Verify the hypervisor is configured with dedicated logical separation (e.g., separate VLANs, virtual switches, or firewall rules) for administrative traffic and guest VM data traffic, even if sharing a single physical NIC.
2. From the Guest VM, attempt Man-in-the-Middle (MITM) and spoofing attacks targeting the administrative network subnet, aiming to intercept or masquerade as administrative traffic.
3. Examine the virtual network configuration to confirm that the separation rules are enforced at the highest privilege level (hypervisor or dedicated virtual switch).

Expected Result: Guest traffic is unable to interfere with, snoop on, or spoof packets on the dedicated administrative logical network path.

Step 3 – Advanced (REQ-H-CP-ISO-003)

1. Confirm through examination that the administrative interfaces are segregated using either dedicated physical network interfaces OR by requiring end-to-end cryptographic separation.
2. If cryptographic separation is used, conduct an analysis of the cryptographic key management and certificate validation process to ensure standard security practices are followed.
3. Attempt to intercept administrative traffic between the management terminal and the hypervisor host and confirm the payload is undecipherable and cannot be manipulated due to encryption and authentication.

Expected Result: Administrative interfaces are protected by demonstrable physical or cryptographic controls, preventing lateral movement or eavesdropping from the guest environment.

Assignment of Verdict

The final verdict is assigned based on the highest level successfully verified during the assessment. The assessment activities are cumulative.

- PASS for Basic: Requires Step 1 to pass.
- PASS for Elevated: Requires Step 1 AND Step 2 to pass.
- PASS for Advanced: Requires Step 1 AND Step 2 AND Step 3 to pass.
- FAIL Condition: The overall assessment for a target level is FAIL if any required step for that level (or any preceding step) fails.

Supporting Evidence

REQ-H-CP-ISO-001 (Basic): Test logs demonstrating rejected access attempts to administrative interfaces in Step 1.

REQ-H-CP-ISO-002 (Elevated): All evidence required for Basic, and: Network configuration files and logs proving the existence and enforcement of dedicated logical separation (VLANs/firewall rules) in Step 2. Test logs showing resistance to MITM/spoofing attacks.

REQ-H-CP-ISO-003 (Advanced): All evidence required for Elevated, and: Vendor documentation detailing physical separation or the cryptographic ciphers and key management used in Step 3. Test logs confirming encryption effectiveness against interception attempts.

13.4 Assessment for Network Plane Separation (A-NP-ISO)

Assessment Reference

- REQ-H-NP-ISO-001 (Basic)
- REQ-H-NP-ISO-002 (Elevated)
- REQ-H-NP-ISO-003 (Advanced)

Assessment Objective

Verify that the hypervisor enforces effective network separation between critical traffic types (management, host, guest) using logical mechanisms, access controls, and, at the highest level, physical or cryptographic isolation to prevent unauthorized cross-plane access or interference.

Assessment Preparation

REQ-H-NP-ISO-001 (Basic): Configure a test environment with at least one Guest VM, the Hypervisor under test, and an external management terminal. Ensure access to the hypervisor's network configuration documentation, including virtual switch settings and VLAN/VXLAN definitions.

REQ-H-NP-ISO-002 (Elevated): All items required for Basic, and: Ensure access to the hypervisor's Access Control List (ACL) configuration engine and integrated firewall settings.

REQ-H-NP-ISO-003 (Advanced): All items required for Elevated, and: Confirm the use of a dedicated management network interface or obtain documentation for the cryptographic mechanism used for tunnel separation (if applicable).

Assessment Activities

Step 1 – Basic (REQ-H-NP-ISO-001)

1. Review the virtual networking configuration (e.g., virtual switches, port groups, VLAN/VXLAN tags) to confirm that dedicated logical segments have been created for: management, host, and guest network traffic.
2. Perform network scans and passive monitoring from one Guest VM. Attempt to discover IP addresses or services belonging to the management and host segments.
3. From the host operating system, attempt to communicate with network services running on the guest network segment without using the hypervisor's intended communication path.

Expected Result: The hypervisor successfully enforces logical segregation of traffic planes. Unauthorized communication and service discovery attempts across the logical boundaries fail.

Step 2 – Elevated (REQ-H-NP-ISO-002)

1. Review the hypervisor's integrated firewall or virtual switch filtering rules to confirm the implementation of explicit Access Control Lists (ACLs) that strictly block traffic from the Guest plane to the Management plane.
2. From a compromised Guest VM, attempt to send spoofed or malicious packets directly targeting management interfaces, overriding the logical separation established in Step 1.
3. Attempt to flood the Guest network segment to verify that the hypervisor's traffic shaping/filtering mechanisms prevent the congestion from spreading to the host or management segments (DoS protection).

Expected Result: The hypervisor successfully enforces ACLs and filtering rules. Active attempts to breach the logical boundaries are blocked by the filtering layer, preventing cross-plane traffic.

Step 3 – Advanced (REQ-H-NP-ISO-003)

1. Verify that the administrative management network is isolated using either a dedicated physical network interface that is unshared with guest traffic, OR by implementing cryptographic isolation for all management communications.
2. If cryptographic isolation is used, attempt passive decryption of intercepted management traffic to confirm the integrity and confidentiality of the communication channel.
3. Review the operational procedures to ensure physical/cryptographic separation is maintained during maintenance and monitoring activities.

Expected Result: Management traffic is demonstrably protected by physical or cryptographic separation, effectively mitigating advanced lateral movement threats originating from other planes.

Assignment of Verdict

The final verdict is assigned based on the highest level successfully verified during the assessment. The assessment activities are cumulative.

- PASS for Basic: Requires Step 1 to pass.
- PASS for Elevated: Requires Step 1 AND Step 2 to pass.
- PASS for Advanced: Requires Step 1 AND Step 2 AND Step 3 to pass.
- Fail Condition: The overall assessment for a target level is FAIL if any required step for that level (or any preceding step) fails.

Supporting Evidence

REQ-H-NP-ISO-001 (Basic): Test logs proving the failure of cross-plane discovery and basic communication attempts in Step 1. Network configuration documentation confirming logical segregation.

REQ-H-NP-ISO-002 (Elevated): All evidence required for Basic, and: Network configuration files and firewall/ACL rule sets confirming the implementation of traffic filtering policies in Step 2. Test logs proving resistance to spoofing/flooding attacks.

REQ-H-NP-ISO-003 (Advanced): All evidence required for Elevated, and: Physical network diagrams or cryptographic configuration evidence used for the management plane in Step 3. Test logs confirming encryption effectiveness against interception.

13.5 Assessment for Boot Chain Integrity Verification (A-BT-VER)

Assessment Reference

- REQ-H-B-INT-001 (Basic)
- REQ-H-B-INT-002 (Elevated)
- REQ-H-B-INT-003 (Advanced)

Assessment Objective

To verify and document that the Hypervisor under test implements the chosen mechanism (hashing, chained signatures, or hardware root of trust) to ensure the integrity of its critical boot components before initializing guest workloads.

Assessment Preparation

REQ-H-B-INT-001 (Basic)

1. Obtain and analyze the Manufacturer's technical documentation detailing the boot process and hashing integrity mechanism.
2. Prepare the test environment with access to the hypervisor's kernel or microkernel binary for controlled corruption attempts.

REQ-H-B-INT-002 (Elevated)

1. Obtain and analyze the Manufacturer's technical documentation detailing the chained signature mechanism and cryptographic configuration.
2. Prepare the test environment with access to all boot components (bootloader, kernel, modules) for replacement attempts.
3. Ensure access to the cryptographic key management policy used for component signing.

REQ-H-B-INT-003 (Advanced)

1. Obtain and analyze the Manufacturer's technical documentation detailing the Hardware Root of Trust (HrOT) implementation and the secure update mechanism.
2. Prepare the test environment with specialized tools for attempting fault injection, external debugging, or unauthorized firmware modification.
3. Ensure access to the cryptographic key management policy and the HrOT configuration logs (e.g., TPM Platform Configuration Registers or equivalent).

Assessment Activities**Section A: Basic (REQ-H-B-INT-001: Hashing Verification)**

1. Verify documentation confirming the Hypervisor computes a cryptographic hash (e.g., SHA-256) of the kernel or microkernel.
2. Isolate and corrupt the hypervisor kernel/microkernel binary by making a byte-level modification.
3. Initiate a boot attempt with the corrupted binary.
4. Observe and record the system state to confirm the boot halt was triggered by the integrity check failure.

Expected Result: The boot process is successfully halted and guest workloads are prevented from initiating, confirming integrity failure detection.

Section B: Elevated (REQ-H-B-INT-002: Chained Signatures)

1. Verify documentation confirming the use of a digital signature-based chained trust mechanism across boot components (e.g., bootloader validates kernel).
2. Replace a signed boot component with a functionally similar component that is signed by an unauthorized, untrusted private key.
3. Initiate a system boot attempt with the unauthorized component.
4. Review the Manufacturer's key management policy to confirm adherence to best practices for securing the trusted signing root.

Expected Result: The hypervisor fails the digital signature validation against its trusted certificate root, rejecting the component and halting the boot process prior to guest workload exposure.

Section C: Advanced (REQ-H-B-INT-003: Hardware Root of Trust)

1. Verify documentation confirming the integrity verification process is anchored to a Hardware Root of Trust (HrOT) for validating the firmware and bootloader stages.
2. Attempt to break the chain of trust using recognized techniques (e.g., fault injection, unauthorized firmware flashing, external debugging) targeting the trust anchor or initial measured components.
3. Analyze the boot chain log (e.g., PCR values) to confirm that all subsequent components (kernel, management services) were measured and cryptographically validated by the chain initiated by the HrOT.

Expected Result: The Hypervisor under test successfully detects the attempted compromise of the trust chain, preventing system boot and entering a secure, uncompromised recovery state.

Assignment of Verdict

The final verdict is assigned based solely on the successful completion of the Assessment Activities required for the **target certification level** chosen by the Manufacturer.

- PASS for REQ-H-B-INT-001 (Basic): Requires Section A to pass.
- PASS for REQ-H-B-INT-002 (Elevated): Requires Section B to pass.
- PASS for REQ-H-B-INT-003 (Advanced): Requires Section C to pass.

- Fail Condition: If the required section (A, B, or C) fails, the assessment for that specific requirement is FAIL.

Supporting Evidence

REQ-H-B-INT-001 (Basic): Test logs demonstrating the failure of the corrupted component to load. Documentation verifying the hashing algorithm and its implementation.

REQ-H-B-INT-002 (Elevated): Test logs confirming the rejection of the unauthorized signature. Documentation detailing the signature validation process, the trust chain, and the associated key management policy.

REQ-H-B-INT-003 (Advanced): Test reports and logs proving the integrity mechanisms successfully defended against physical/logical tampering attempts. Documentation detailing the HRoT configuration and the analysis of the boot chain log (e.g., PCR measurement records).

Annex A (informative): CRA Mapping Table

[To be completed in a future version]

Commented [LP66]: Should come first - as Annex A.

Commented [MH67R66]: Yes, moved to Annex A

Harmonised Standard ETSI EN <NUMBER>					
Requirement				Requirement Conditionality	
No	Description	Requirements of Regulation	Clause(s) of the present document	U/C	Condition
1	Products with digital elements shall be designed, developed and produced in such a way that they ensure an appropriate level of cybersecurity based on the risks.	Annex I, Part I, (1)	Hypervisor: 10.1.1 10.1.2 10.1.3 10.1.4 10.1.5 10.1.6 CRS: 11.1.1 11.1.2 11.1.3 11.1.4 11.1.5 11.1.6	U	-
2	Products with digital elements shall be made available on the market without known exploitable vulnerabilities.	Annex I, Part I, (2)(a)	Hypervisor: 10.1.2 10.1.8 10.1.9 10.1.10 CRS: 11.1.2 11.1.8 11.1.9 11.1.10		
3	Products with digital elements shall be made available on the market with a secure by default configuration, unless otherwise agreed between manufacturer and business user in relation to a tailor-made product with digital elements, including the possibility to reset the product to its original state.	Annex I, Part I, (2)(b)	Hypervisor: 10.1.9 10.1.10 CRS: 11.1.9 11.1.10		
4	Products with digital elements shall ensure that vulnerabilities can be	Annex I, Part I, (2)(c)	Hypervisor: 10.1.8		

Harmonised Standard ETSI EN <NUMBER>					
Requirement				Requirement Conditionality	
No	Description	Requirements of Regulation	Clause(s) of the present document	U/C	Condition
	addressed through security updates, including, where applicable, through automatic security updates that are installed within an appropriate timeframe enabled as a default setting, with a clear and easy-to-use opt-out mechanism, through the notification of available updates to users, and the option to temporarily postpone them.		CRS: 11.1.8		
5	Products with digital elements shall ensure protection from unauthorised access by appropriate control mechanisms, including but not limited to authentication, identity or access management systems, and report on possible unauthorised access	Annex I, Part I, (2)(d)	Hypervisor: 10.1.3 10.1.4 CRS: 11.1.3 11.1.4		
6	Products with digital elements shall protect the confidentiality of stored, transmitted or otherwise processed data, personal or other, such as by encrypting relevant data at rest or in transit by state of the art mechanisms, and by using other technical means.	Annex I, Part I, (2)(e)	Hypervisor: 10.1.5 CRS: 11.1.5		
7	Products with digital elements shall protect the integrity of stored, transmitted or otherwise processed data, personal or other, commands, programs and configuration against any manipulation or modification not authorised by the user, and report on corruptions.	Annex I, Part I, (2)(f)	Hypervisor: 10.1.2 10.1.7 CRS: 11.1.2 11.1.7		
8	Products with digital elements shall process only data, personal or other, that are adequate, relevant and limited to what is necessary in relation to the intended purpose of the product with digital elements (data minimisation).	Annex I, Part I, (2)(g)	Hypervisor: 10.1.11 CRS: 11.1.11		
9	Products with digital elements shall protect the availability of essential and basic functions, also after an incident, including through resilience and mitigation	Annex I, Part I, (2)(h)	Hypervisor: 10.1.6 CRS: 11.1.6		

Harmonised Standard ETSI EN <NUMBER>					
Requirement				Requirement Conditionality	
No	Description	Requirements of Regulation	Clause(s) of the present document	U/C	Condition
	measures against denial-of-service attacks.				
10	Products with digital elements shall minimise the negative impact by the products themselves or connected products on the availability of services provided by other products or networks.	Annex I, Part I, (2)(i)	Hypervisor: 10.1.1 10.1.6 CRS: 11.1.1 11.1.6		
11	Products with digital elements shall be designed, developed and produced to limit attack surfaces, including external interfaces.	Annex I, Part I, (2)(j)	Hypervisor: 10.1.1 10.1.10 CRS: 11.1.1 11.1.10		
12	Products with digital elements shall be designed, developed and produced to reduce the impact of an incident using appropriate exploitation mitigation mechanisms and techniques.	Annex I, Part I, (2)(k)	Hypervisor: 10.1.1 10.1.2 10.1.6 CRS: 11.1.1 11.1.2 11.1.6		
13	Products with digital elements shall provide security related information by recording and monitoring relevant internal activity, including the access to or modification of data, services or functions, with an opt-out mechanism for the user.	Annex I, Part I, (2)(l)	Hypervisor: 10.1.7 10.1.11 CRS: 11.1.7 11.1.11		
14	Products with digital elements shall provide the possibility for users to securely and easily remove on a permanent basis all data and settings and, where such data can be transferred to other products or systems, ensure that this is done in a secure manner.	Annex I, Part I, (2)(m)	Hypervisor: 10.1.5 10.1.9 CRS: 11.1.5 11.1.9		
...	CRA Annex I – Part II: Vulnerability Handling Requirements		12.1		

Key to columns:**Requirement:**

No A unique identifier for one row of the table which may be used to identify a requirement.

Description A textual reference to the requirement.

Requirements of Regulation

Identification of article(s) defining the requirement in the Regulation.

Clause(s) of the present document

Identification of clause(s) defining the requirement in the present document unless another document is referenced explicitly.

Requirement Conditionality:

U/C	Indicates whether the requirement is unconditionally applicable (U) or is conditional upon the manufacturer's claimed functionality of the equipment (C).
Condition	Explains the conditions when the requirement is or is not applicable for a requirement which is classified "conditional".

Presumption of conformity stays valid only as long as a reference to the present document is maintained in the list published in the Official Journal of the European Union. Users of the present document should consult frequently the latest list published in the Official Journal of the European Union.

Other Union legislation may be applicable to the product(s) falling within the scope of the present document.

Annex B (informative): Illustrative Cybersecurity Risk Assessment Methodology

This annex is informative and non-binding. It provides a risk assessment methodology used in this standard for assessing the cybersecurity risk of a product based on its intended use case and deployment environment. This approach helps determine the risk level for various use cases, guiding the application of appropriate security requirements.

Manufacturers can use this methodology as guidance or apply their own established risk assessment methodologies, provided the outputs meet the criteria of the CRA and conform to prEN XXXXX:20YY (PT1), Clause 5.6.

The risk assessment methodology used herein is a semi-quantitative approach, combining qualitative analysis with numerical scoring. It provides a framework for:

- Identifying product cybersecurity threats.
- Determining the likelihood and magnitude of impact if those threats materialize.
- Evaluating the overall cybersecurity risk for the product within its specific use case and deployment environment.

This methodology aligns with prEN XXXXX:20YY (PT1) general principles on the assessment of product cybersecurity risk, clause 5.6.

B.1 Risk Assessment Process

The risk assessment process focuses on evaluating the product within its defined use case and deployment environment, following a logical sequence of steps:

1. Define the product and its context: Identify the product being assessed, its intended use case, and the specific deployment environment it's intended for. This provides the foundational context for the assessment.
2. Identify product threats: Identify potential cybersecurity threats that could exploit vulnerabilities.

For each identified threat, the following steps should be performed:

3. Analyse risk factors for likelihood and impact as defined in Annex B.2: This step determines the product's cybersecurity risk by evaluating its characteristics related to potential threats. For each identified threat, and for every relevant risk factor, the following guiding question should be applied: *"How does this factor influence the likelihood or impact if the threat were to occur?"*. Each threat is evaluated against a defined set of risk factors covering both Likelihood and Impact. These factors are scored on a three-level scale (Low = 1, Medium = 2, High = 3) using the definitions provided in the risk factor table in Annex B.2.

The analysis involves the following sub-steps:

- Identifying key risk factors: Determine the main aspects of the product's operation within that environment that influence its cybersecurity risk (e.g., connectivity, administrative context, configuration volatility, impact of compromise, integration with critical systems). These include factors that affect either:
 - The likelihood of threat exploitation, or
 - The severity of consequences if the threat materializes.

These factors serve as proxies for the likelihood of a threat materializing and the magnitude of its impact.

- Defining scoring: Assign numerical scores (1 for low risk, 2 for medium, 3 for high) to each risk factor based on how it influences the likelihood of a threat event and the magnitude of its potential impact.
- Derive overall likelihood score: Sum the scores of all risk factors identified as influencing "Likelihood". Map this total score to the Likelihood level (1-4) using the thresholds defined in Annex B.3.1.
 - Derive overall impact score: Sum the scores of all risk factors identified as influencing "Impact". Map this total score to the Impact level (1-4) using the thresholds defined in Annex B.3.2.
 - Determine final risk level: Using the derived overall Likelihood score and overall Impact score, consult the risk matrix (Annex B.4) to determine the final, overall risk level (Low, Medium, High, Critical) for the specific use case.
 - Document and review: Record all assessment findings, including the rationale for scores and the determined risk level.

B.2 Risk Factor Scoring

The following table provides the detailed scoring rubric to be used for assessing cybersecurity risks of products within their use cases and deployment environments. Each risk factor is explicitly linked to whether it primarily influences likelihood or impact. The weight assigned to each main risk factor is used in the calculation of the overall likelihood and impact scores.

Tag	Factor	Type	Description	Low (1)	Medium (2)	High (3)
EXT	External Network Exposure	Likelihood	Measures how reachable the VES/CES is from untrusted networks, including public internet and cross-tenant networks. Higher exposure increases the probability of remote exploitation.	Internal-only access; management on isolated VLAN/bastion; strict firewall segmentation.	Limited/controlled external access (VPN, allow-list, jump host).	Public internet-facing endpoints or broad cross-tenant network paths.
TEN	Tenant Multiplicity & Trust	Likelihood	Captures the number of tenants and their trust level. Multi-tenancy with untrusted tenants increases the chance of malicious insiders or cross-tenant attacks.	Single trusted tenant.	Few semi-trusted tenants.	Many untrusted tenants (public/multi-organisation).
CTL	Control-Plane Exposure & Authorisation	Likelihood	Evaluates the attack surface and security posture of management and orchestration	Local-only or management VLAN via bastion; MFA + RBAC least-privilege; no	Internal network access; SSO/RBAC with moderate admin base.	Internet- or cross-tenant-reachable; weak RBAC; shared or overprivileged accounts.

			interfaces (API, CLI, web UI). Weak access controls here can lead to full system compromise.	shared admin credentials.		
SUP	Update & Supply-Chain Assurance	Likelihood	Assesses the security of update mechanisms, software/image sources, and supply-chain validation. Weak assurance can allow malicious updates or images.	Signed/verified only; private mirrors; staged/canary updates.	Vendor-signed updates/images; occasional direct pulls.	Direct pulls from public registries/vendor repositories; unsigned allowed.
PRV	Privileged Footprint & Hardening	Likelihood	Measures the size and hardening of privileged code in the trusted computing base. A larger or less hardened footprint increases exploitability.	Minimal kernel/drivers; hardened (ASLR, SMEP, SMAP, CET); rootless containers.	Standard builds; mixed hardening; some privileged workloads.	Many privileged processes/drivers; passthrough devices; weak or no hardening.
CHG	Change/Workload Volatility	Impact	Evaluates how frequently the configuration and workloads change. Higher volatility increases the likelihood of misconfigurations or unpatched instances.	Static, rarely changes.	Periodic changes (weekly/monthly).	Highly dynamic (daily/hourly), autoscaling, frequent reconfiguration.
ISO	Isolation Breakout Consequence	Impact	Assesses the damage if the VM/container isolation boundary is broken. Higher values mean more severe lateral movement or system compromise.	No sensitive assets exposed; minimal lateral movement possible.	Exposure of internal systems/data; contained blast radius.	Cross-tenant or critical system compromise; admin plane reachable.
DAT	Data/Safety/Criticality	Impact	Measures the sensitivity of the data and any safety or regulatory implications of compromise. Higher scores indicate severe operational, safety, or compliance impacts.	Public or low-sensitivity data; no safety impact.	Confidential/business data; moderate operational disruption.	Regulated/PII/financial/eHealth data; safety hazard or critical infrastructure outage.
ORB	Orchestration Dependency & Blast Radius	Impact	Evaluates how much damage an orchestrator compromise could cause.	Stand-alone host; no orchestrator or	Basic orchestrator; tenant- or namespace-scoped control.	Full control of critical workloads across multi-node clusters/sites.

			Higher values reflect large-scale or critical workload control.	read-only integration.		
DRC	Detection & Recovery Capability	Impact	Measures the ability to detect, contain, and recover from a compromise. Poor detection/recovery increases the residual impact of an incident.	Centralised logging/alerting; immutable backups; tested recovery plans.	Adequate logging/backups; periodic recovery tests.	Minimal logging/alerting; weak or untested recovery procedures.

B.3 Calculation and Mapping to Likelihood/Impact Levels

This clause defines how the total scores from the risk factor scoring table (Annex B.2) are translated into the 1–3 Likelihood and Impact levels used in the risk matrix (Annex B.4).

B.3.1 Likelihood Score

The risks factors table in Annex B.2 defines five Likelihood factors:

- EXT (External Network Exposure)
- TEN (Tenant Multiplicity & Trust)
- CTL (Control-Plane Exposure & Authorisation)
- SUP (Update & Supply-Chain Assurance)
- PRV (Privileged Footprint & Hardening)

Each factor is scored from 1 (Low) to 3 (High).

- Maximum possible Likelihood score: 5 factors $\times$ 3 = 15
- Minimum possible Likelihood score: 5 factors $\times$ 1 = 5

The resulting total score is mapped to three levels as follows:

Total Likelihood Score Range	Likelihood Level	Description
5 – 8	Low (1)	The event is highly unlikely to occur under normal circumstances.
9 – 12	Medium (2)	The event may occur occasionally in enterprise or cloud deployments.
13 – 15	High (3)	The event is expected to occur or has a high probability of occurring.

B.3.2 Impact Score

The risks factors table in Annex B.2 defines five Impact factors:

- CHG (Change/Workload Volatility)
- ISO (Isolation Breakout Consequence)
- DAT (Data/Safety/Criticality)
- ORB (Orchestration Dependency & Blast Radius)
- DRC (Detection & Recovery Capability)

Each factor is scored from 1 (Low) to 3 (High).

- Maximum possible Impact score: 5 factors $\times$ 3 = 15
- Minimum possible Impact score: 5 factors $\times$ 1 = 5

The resulting total score is mapped to three levels as follows:

Total Impact Score Range	Impact Level	Description
5 – 8	Low (1)	Minimal disruption. No significant loss of confidentiality, integrity, or availability. No impact on safety or critical infrastructure.
9 – 12	Medium (2)	Noticeable disruption requiring recovery. Moderate impact on confidentiality, integrity, or availability. Potential for indirect impact on safety or critical infrastructure.
13 – 15	High (3)	Severe disruption, service interruption, or data loss. Major impact on confidentiality, integrity, or availability. May directly affect safety or critical infrastructure.

B.4 Risk Matrix

The final overall risk level is determined by combining the calculated likelihood level (Annex B.3.1) and impact level (Annex B.3.2) using the matrix below:

Impact \ Likelihood	1 = Low	2 = Medium	3 = High
3 = High	Medium	High	High
2 = Medium	Low	Medium	High
1 = Low	Low	Low	Medium

B.5 Risk Methodology Applied to Use Cases

This clause illustrates the application of the methodology to products deployed within the VES use cases (UC-V1, UC-V2, UC-V3) and CES use cases (UC-C1, UC-C2, UC-C3). The use cases are defined in clause 5 of this standard, while the corresponding threats applicable to these products are described in clause 6.

B.5.1 Risk Evaluation of VES use cases

Hypervisor Type I

T-HYPI-INTF-ESC: Inter-VM interference / VM escape

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	In an isolated host, exploitation requires physical/local access, and workloads are non-critical. Escape doesn't propagate.
V2	1	2	3	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	In enterprise clusters, centralised management and more workloads make escape feasible, exposing all VMs of the tenant.
V3	3	3	3	3	3	3	3	3	3	3	15 → High	15 → High	High	In clouds, shared hypervisors host untrusted tenants. Escape allows lateral compromise across organisations and critical data.

T-HYP1-INTEG-COMP: Integrity compromise of workloads / hypervisor components

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	Tampering is possible but limited to a single low-value host, so consequences are minor.
V2	1	2	3	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	Compromised hypervisor files in enterprises could give persistent control over business workloads.
V3	3	3	3	3	3	3	3	3	3	3	15 → High	15 → High	High	At cloud scale, compromise of binaries or configs undermines all tenants and platform trust.

T-HYP1-UNAUTH-ADMIN: Unauthorised access to management interfaces

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	Local admin interfaces may be weakly protected, but lack of internet access limits real exposure.
V2	1	2	3	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	In enterprises, centralised management is a key target. Compromise gives total control over tenant workloads.
V3	3	3	3	3	3	3	3	3	3	3	15 → High	15 → High	High	In multi-tenant clouds, exposed admin APIs are critical. Breach means compromise across organisations.

T-HYP1-UNAUTH-FUNC: Unauthorised privileged actions

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	Privilege abuse is possible, but isolated and non-sensitive workloads keep risk low.
V2	1	2	3	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	Weak RBAC in enterprise setups could escalate low-privilege users to admin, exposing workloads.
V3	2	3	3	3	3	3	3	3	3	3	14 → High	15 → High	High	In clouds, weak authorisation lets attackers perform privileged actions across tenants, endangering sensitive data.

T-HYP1-DATA-DISC: Data disclosure

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	1	1	1	1	1	2	6 → Low	6 → Low	Low	Data is public or low-value, so even if exposed the impact is negligible.
V2	2	2	2	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	Enterprises often handle sensitive business data. Disclosure would harm confidentiality but not safety.

V3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	In clouds, regulated data (finance, health, gov) is at stake. Disclosure across tenants is severe.
----	---	---	---	---	---	---	---	---	---	---	-----------	-----------	------	--

T-HYPI-AVAIL-DOS: DoS via resource exhaustion

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	1	1	1	1	1	1	1	5 → Low	5 → Low	Low	Resource exhaustion only affects local, non-critical services.
V2	1	2	2	2	2	2	2	2	2	2	9 → Med	10 → Med	Medium	In enterprises, DoS disrupts workloads and operations, but only one tenant is affected.
V3	2	3	2	2	3	3	3	2	3	2	12 → Med	13 → High	High	In multi-tenant clouds, "noisy neighbour" effects cascade, degrading availability across many customers.

T-HYPI-LOG-INT: Insufficient logging / log tampering

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	1	1	1	1	1	1	2	5 → Low	6 → Low	Low	Weak logging hides small incidents, but effect is minimal in isolated devices.
V2	1	2	2	2	2	2	2	2	2	2	9 → Med	10 → Med	Medium	In enterprises, poor logs delay detection of attacks on multiple workloads.
V3	2	3	3	2	2	2	3	3	3	3	12 → Med	14 → High	High	In clouds, forensic gaps cripple incident response. Breaches remain hidden across tenants, amplifying impact.

T-HYPI-UPD-COMP: Malicious or compromised update

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	2	2	1	1	1	1	2	7 → Low	6 → Low	Low	Updates may be poorly managed, but compromise only affects one non-critical host.
V2	2	2	2	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	Enterprises face patch delays. A poisoned update could affect all workloads of the tenant.
V3	2	3	2	3	3	2	3	3	3	3	13 → High	14 → High	High	In clouds, malicious updates scale to all tenants. Even with monitoring, blast radius is global.

T-HYPI-CFG-COMP: Unauthorised or insecure configuration changes

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	1	7 → Low	5 → Low	Low	Local config errors weaken isolation but remain local and low-impact.

V2	2	2	3	2	2	2	2	2	2	2	11 → Med	10 → Med	Medium	Misconfigurations in enterprises can expose management or data, raising risk significantly.
V3	2	3	3	2	3	3	3	3	3	3	13 → High	14 → High	High	In clouds, insecure configuration spreads across tenants, risking systemic compromise.

T-HYP1-DEF-COMP: Insecure default configuration

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	1	7 → Low	5 → Low	Low	Defaults may be weak, but isolated systems are less exposed.
V2	2	2	2	2	2	2	2	2	2	2	10 → Med	10 → Med	Medium	In enterprises, insecure defaults like open ports can be abused internally, affecting all workloads.
V3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	In clouds, default accounts or open services are immediately exploitable at scale.

T-HYP1-SC-COMP: Supply chain compromise

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	2	2	1	1	1	1	2	7 → Low	6 → Low	Low	A backdoored driver would affect the local host only; workloads are low value.
V2	2	2	2	3	2	2	2	2	2	2	11 → Med	11 → Med	Medium	In enterprises, compromised components introduce persistent backdoors that affect all workloads.
V3	2	3	2	3	3	2	3	3	3	3	13 → High	14 → High	High	In clouds, a poisoned library in the hypervisor affects every tenant. The systemic impact is severe.

Hypervisor Type II**T-HYP2-HOST-ABUSE: Guest-to-host escalation**

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	2	7 → Low	6 → Low	Low	Attacks on the host OS are possible, but isolated devices run low-value workloads with minimal connectivity.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	In enterprise, abuse of host OS services (e.g., file sharing) could compromise all internal workloads.
V3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	Public multi-tenant environments are especially exposed; host abuse undermines VM isolation and risks regulated data.

T-HYP2-INTEG-COMP: Integrity compromise of hosted hypervisor

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	2	7 → Low	6 → Low	Low	Local tampering of binaries or images affects only a single host with trivial workloads.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	If enterprise host OS or hypervisor binaries are compromised, all tenant workloads are exposed.
V3	3	3	3	3	3	3	3	3	3	3	15 → High	15 → High	High	Supply chain or filesystem tampering at cloud scale undermines trust in all workloads and services.

T-HYP2-UNAUTH-ADMIN: Unauthorized admin access

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	1	1	1	1	1	1	6 → Low	5 → Low	Low	Default or weak creds may exist, but impact is limited to a single machine.
V2	2	2	3	2	2	2	2	2	2	2	11 → Medium	10 → Medium	Medium	Weak authentication in enterprise hypervisors/host OS admin accounts could lead to full outage.
V3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	Internet-facing Type-2 consoles or host OS admin accounts give attackers systemic control in cloud.

T-HYP2-UNAUTH-FUNC: Unauthorized privileged actions

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	1	7 → Low	5 → Low	Low	Misuse of local runtime functions is possible but workloads/data are non-critical.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	In enterprise setups, insufficient authorisation enables unauthorised privileged actions across workloads.
V3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	Multi-tenant Type-2 hypervisors with weak RBAC allow broad privilege abuse and tenant data exposure.

T-HYP2-DATA-DISC: Sensitive data disclosure

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	1	1	1	1	1	1	1	5 → Low	5 → Low	Low	Data on isolated systems is low-sensitivity, so leaks have little consequence.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Enterprise CES hosts confidential business data, making disclosure significant but tenant-limited.
V3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	In cloud environments, disclosure impacts

																regulated/critical workloads across tenants.
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

T-HYP2-AVAIL-DOS: Resource exhaustion

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	1	1	1	1	1	1	1	5 → Low	5 → Low	Low	DoS causes only local disruption of non-critical services.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Enterprise workloads may be disrupted but effect is tenant-contained.
V3	3	3	2	2	3	3	3	3	3	2	13 → High	14 → High	High	DoS in multi-tenant clouds disrupts availability broadly (e.g., noisy neighbour).

T-HYP2-LOG-INT: Logging weakness/tampering

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	1	1	1	1	1	1	2	6 → Low	6 → Low	Low	Log tampering is possible but low-value workloads mean little impact.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	In enterprise contexts, weak logs delay detection and prolong dwell time of attackers.
V3	2	3	3	2	3	2	3	3	3	3	13 → High	14 → High	High	In multi-tenant clouds, insufficient logging makes forensic analysis impossible, magnifying impact.

T-HYP2-UPD-COMP: Malicious/compromised update

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	Malicious updates affect only a single isolated Type-2 host.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Enterprise admins may delay patches or fetch them manually; compromised updates could expose all workloads.
V3	2	3	2	3	3	2	3	3	3	3	13 → High	14 → High	High	Poisoned installers at cloud scale compromise multi-tenant trust.

T-HYP2-CFG-COMP: Insecure configuration changes

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	1	7 → Low	5 → Low	Low	Local misconfigurations are limited to one machine.
V2	2	2	3	2	2	2	2	2	2	2	11 → Medium	10 → Medium	Medium	Insecure configs in enterprise Type-2 hypervisors (e.g., broad mounts) enable escalation across workloads.

V3	2	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	Multi-tenant Type-2 configs (e.g., open shared folders) expose many tenants at once.
----	---	---	---	---	---	---	---	---	---	---	-----------	-----------	------	--

T-HYP2-DEF-COMP: Insecure defaults

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	2	1	2	1	1	1	1	1	7 → Low	5 → Low	Low	Defaults like open guest-host shares are dangerous, but isolated non-critical setups reduce impact.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Insecure defaults (e.g., shared folders, network) in enterprise Type-2 setups compromise workloads.
V3	3	3	3	2	3	3	3	3	3	3	15 → High	14 → High	High	Defaults in multi-tenant Type-2 hypervisors allow attackers immediate leverage in public clouds.

T-HYP2-SC-COMP: Supply chain compromise

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
V1	1	1	1	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	Malicious dependencies affect only a single isolated Type-2 host.
V2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Enterprises risk compromise of the host OS + hypervisor stack from poisoned dependencies.
V3	2	3	2	3	3	2	3	3	3	3	13 → High	14 → High	High	In public CES/Type-2, poisoned upstream drivers or libraries propagate across many tenants.

B.5.2 Risk Evaluation of CES use cases**CRS****T-CRS-INTF-ESC: Runtime-level container breakout**

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	2	1	2	1	1	1	1	2	7 → Low	6 → Low	Low	Breakout is possible on an isolated device but workloads are non-critical and exposure is minimal.
C2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Enterprise CES hosts sensitive internal workloads. Escape compromises all apps of the tenant.
C3	3	3	3	3	3	3	3	3	3	3	15 → High	15 → High	High	In multi-tenant public CES, escape exposes untrusted tenants, regulated data, and may cascade cluster-wide.

T-CRS-INTEG-COMP: Runtime-level tampering

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	2	1	2	1	1	1	1	2	7 → Low	6 → Low	Low	Tampering may disable isolation but only affects one isolated device with low-value data.
C2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Compromise of runtime configs in enterprise CES undermines security controls across the organisation's workloads.
C3	3	3	3	3	3	3	3	3	3	3	15 → High	15 → High	High	At cloud scale, tampered runtimes evade orchestrator controls and weaken separation across all tenants.

T-CRS-UNAUTH-ADMIN: Unauthorized runtime control access

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	2	1	1	1	1	1	1	1	6 → Low	5 → Low	Low	Local misuses of runtime binaries are possible but isolated and low-impact.
C2	2	2	3	2	2	2	2	2	2	2	11 → Medium	10 → Medium	Medium	Internal attackers could bypass orchestrator APIs and directly control runtimes, affecting enterprise workloads.
C3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	In public clouds, exposed runtime sockets or binaries give attackers deep control across tenant workloads.

T-CRS-UNAUTH-FUNC: Unauthorized privileged ops

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	2	1	2	1	1	1	1	2	7 → Low	6 → Low	Low	Privilege abuse is possible but isolated containers are low-value and local.
C2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Weak RBAC allows escalation inside enterprise CES, potentially exposing sensitive apps.
C3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	In clouds, inadequate runtime authorisation enables attackers to manipulate untrusted tenants' containers.

T-CRS-DATA-DISC: Data disclosure via runtime

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	1	1	1	1	1	1	1	1	5 → Low	5 → Low	Low	Low-sensitivity data (e.g., telemetry) means disclosure has minimal consequences.
C2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Confidential enterprise data could be exposed if runtime isolation fails.
C3	3	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	Cloud CES may host financial, health, or government data. Runtime

C3	2	3	3	2	3	3	3	3	3	3	14 → High	14 → High	High	Public CES misconfigs (e.g., open APIs, weak seccomp) enable widespread tenant compromise.
----	---	---	---	---	---	---	---	---	---	---	-----------	-----------	------	--

T-CRS-DEF-COMP: Insecure defaults

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	2	1	2	1	1	1	1	1	7 → Low	5 → Low	Low	Defaults like no seccomp/cgroups matter little in isolated low-value setups.
C2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Insecure defaults (e.g., broad mounts) in enterprise CES affect all workloads.
C3	3	3	3	2	3	3	3	3	3	3	15 → High	14 → High	High	Defaults in public CES (no AppArmor, unbounded cgroups) expose tenants immediately at scale.

T-CRS-SC-COMP: Supply chain compromise

UC	EXT	TEN	CTL	SUP	PRV	CHG	ISO	DAT	ORB	DRC	Likelihood (sum/level)	Impact (sum/level)	Risk	Rationale
C1	1	1	1	2	2	1	1	1	1	2	8 → Low	6 → Low	Low	Poisoned runtime libraries only compromise one isolated host with low-sensitivity data.
C2	2	2	2	2	2	2	2	2	2	2	10 → Medium	10 → Medium	Medium	Enterprises risk persistence across internal workloads when runtime dependencies are compromised.
C3	2	3	2	3	3	2	3	3	3	3	13 → High	14 → High	High	In public CES, a poisoned dependency embedded in runtimes propagates to all tenants and workloads.

Annex C (normative):

Threat-to-Requirement Traceability Matrix

C.1 VES

[To be completed in a future version]

Hypervisor Type I

Threat ID	Basic Requirements	Elevated Requirements	Advanced Requirements
T-HYP1-INTF-ESC	REQ-H-VM-ISO-001	REQ-H-VM-ISO-002	REQ-H-VM-ISO-003
	REQ-H-CP-ISO-001	REQ-H-CP-ISO-002	REQ-H-CP-ISO-003
	REQ-H-NP-ISO-001	REQ-H-NP-ISO-002	REQ-H-NP-ISO-003
T-HYP1-INTEG-COMP	REQ-H-B-INT-001	REQ-H-B-INT-00	REQ-H-B-INT-003

	REQ-H-IMG-INT-001	REQ-H-IMG-INT-002 REQ-H-RP-INT-001	REQ-H-IMG-INT-003 REQ-H-RP-INT-002 REQ-H-RA-INT-001
T-HYP1-UNAUTH-ADMIN	REQ-H-ADMIN-AUTH-001 REQ-H-SERV-AUTH-001	REQ-H-ADMIN-AUTH-002 REQ-H-SERV-AUTH-002	REQ-H-ADMIN-AUTH-003 REQ-H-SERV-AUTH-003
T-HYP1-UNAUTH-FUNC	REQ-H-ADMIN-AUTHZ-001 REQ-H-SERV-AUTHZ-001	REQ-H-ADMIN-AUTHZ-002 REQ-H-SERV-AUTHZ-002	REQ-H-ADMIN-AUTHZ-003 REQ-H-SERV-AUTHZ-003
T-HYP1-DATA-DISC			
T-HYP1-AVAIL-DOS			
T-HYP1-LOG-INT			
T-HYP1-UPD-COMP			
T-HYP1-CFG-COMP			
T-HYP1-DEF-COMP			
T-HYP1-SC-COMP			

C.2 CES

[To be completed in a future version]

Annex D (informative):
Illustrative implementation security controls

This annex provides illustrative controls that manufacturers may consider when implementing the security requirements defined in clauses 10 and 11 of this standard.

[To be completed in a future version]

D.1 VES

D.1.1 Hypervisor

D.1.1.1 Isolation

D.1.1.1.1 VM Isolation

Implementation Notes

- Type I:
 - Basic: Enforce isolation using CPU privilege levels, software-based memory segmentation and paging, and virtual network segmentation.
 - Elevated: Implement strong isolation using the virtualization mechanisms provided by the platform, such as Intel VT-x / AMD-V or ARM Virtualization Extensions for CPU, IOMMU (Intel VT-d / AMD-Vi) for I/O separation, and SR-IOV for device partitioning. Where hardware support is not available, use equivalent software mechanisms (e.g., paravirtualization).

Commented [LP68]: If it is illustrative, it is informative.

Commented [MH69R68]: Yes, normative removed

- Advanced: Strengthen isolation with mitigations against side-channel attacks (e.g., L1TF, MDS, Spectre/Meltdown) and apply memory protection technologies such as Intel TME or AMD SEV where supported.
- Type II:
 - Basic: Enforce isolation using host OS process and address space separation, and host-managed network namespaces and ACLs.
 - Elevated: Implement strong isolation using the host kernel's virtualization mechanisms, such as KVM or Hyper-V with Intel VT-x / AMD-V, or ARM virtualization extensions. For I/O separation, use IOMMU via host drivers and SR-IOV for device partitioning. Where hardware support is not available, rely on equivalent software mechanisms.
 - Advanced: Strengthen isolation with mitigations against side-channel attacks (e.g., L1TF, MDS, Spectre/Meltdown) and apply memory protection technologies such as Intel TME or AMD SEV where supported.

D.1.1.1.2 Control Plane Isolation

Implementation Notes

- Type I: Directly enforces isolation in the hypervisor's control plane, ensuring management services are on separate logical or physical channels from guest VM traffic.
- Type II: Enforces separation via host OS privilege boundaries, restricting administrative interfaces to privileged host processes and isolating them from guest traffic using host networking controls.

D.1.1.1.3 Network Plane Separation

Implementation Notes

- Type I: Uses built-in vSwitch or hypervisor-controlled networking to segregate planes, apply ACLs, and optionally use dedicated physical NICs for management.
- Type II: Uses host OS-level network namespaces, VLAN tagging, bridge isolation, and firewall rules to separate management and guest networks, with physical separation coordinated via host OS network configuration.

D.1.1.2 Integrity Protection

D.1.1.2.1 Boot chain integrity

Implementation Notes

- Type I:
 - Basic: Use UEFI Secure Boot or coreboot with signature validation of hypervisor kernel.
 - Elevated: Extend Basic with verified boot for bootloader (e.g., GRUB signed binaries) chained to kernel verification.
 - Advanced: Implement full chain-of-trust using TPM 2.0, Intel Boot Guard, or AMD Hardware Verified Boot, including firmware, bootloader, hypervisor kernel, and management service binaries.
- Type II:
 - Basic: Use host OS secure/verified boot (e.g., Linux Kernel Lockdown + module signing) to validate hypervisor kernel module at load.
 - Elevated: Add signature validation of bootloader (e.g., GRUB signed with shim validation) in host OS boot process.
 - Advanced: Use hardware-based root of trust via host platform (e.g., TPM 2.0 + measured boot) covering firmware → bootloader → host kernel → hypervisor components.

D.1.1.2.2 Guest VM image integrity

Implementation Notes

- Type I:

- Basic: Hash verification (e.g., SHA-256) at VM load time; compare with trusted value in hypervisor datastore.
- Elevated: Signature verification (e.g., RSA/ECDSA) against keys in hypervisor key store or host OS.
- Advanced: Keys in TPM/HSM; verification done in firmware-assisted secure context before VM image is mapped.
- Type II:
 - Basic: Hash verification using host OS or hypervisor interface.
 - Elevated: Signature verification using keys in host OS key store.
 - Advanced: TPM/HSM-based key storage with secure boot integration, validation before VM start.

D.1.1.2.3 Runtime integrity protection

For REQ-H-INTEG-003 (Elevated)

Mechanisms that may be used to meet this requirement include:

- Require that all hypervisor or kernel modules are digitally signed and validated before loading.
- Mark privileged memory regions as read-only so they cannot be changed during runtime.
- Use built-in hypervisor or kernel lockdown features to prevent unauthorized modifications, even by privileged accounts.
- Use integrity measurement and logging tools (e.g., Integrity Measurement Architecture (IMA)) to detect and record changes to important files or settings.

For REQ-H-INTEG-003 (Enhanced)

Mechanisms that may be used to meet this requirement include:

- Use a Trusted Platform Module (TPM) or similar to measure runtime state and prevent it from running if tampered.
- Use Dynamic Root of Trust for Measurement (DRTM) to reset the CPU into a known secure state if needed.
- Use hardware protections such as Intel CET or AMD SEV-ES/SNP to block control-flow attacks and unauthorized memory access.
- Deploy hypervisor introspection or runtime verification tools that watch privileged code and memory for suspicious changes.

D.1.1.2.4 Remote attestation

D.1.1.3 Authentication

D.1.1.3.1 Administrative Authentication

Implementation Notes:

Basic (REQ-H-ADMIN-AUTH-001):

- Passwords should follow strong policies: minimum length (≥ 12 characters), complexity rules (upper/lowercase, numbers, symbols), and periodic rotation.

Elevated (REQ-H-ADMIN-AUTH-002):

- SSH key-based authentication is the most widely used mechanism for secure administrative access.
- Recommended key types: Ed25519, ECDSA P-256, or RSA ≥ 3072 bits.
- Private keys should be stored in secure local storage (e.g., OS keyrings) or in hardware tokens to prevent theft or reuse.

Advanced (REQ-H-ADMIN-AUTH-003):

- Mutual TLS, smartcards, or hardware tokens may be used as certificate carriers.

- Hardware security modules (HSMs), Trusted Platform Modules (TPMs), or secure enclaves should be used to securely store and protect administrative credentials.

D.1.1.3.2 Service Authentication

Implementation Notes:

Basic (REQ-H-SERV-AUTH-001):

- Service-to-hypervisor authentication is often implemented using API keys, credentials issued per service account, or JSON Web Tokens (JWTs).

Elevated (REQ-H-SERV-AUTH-002):

- Mutual TLS (mTLS) is the most common mechanism for strong service authentication in hypervisors and cloud platforms.
- TLS 1.3 (preferred) or TLS 1.2 with strong cipher suites (e.g., AES-GCM, ChaCha20-Poly1305, SHA-256/384) should be enforced.
- Weak or legacy ciphers (e.g., RC4, 3DES, MD5) must not be supported.

Advanced (REQ-H-SERV-AUTH-003):

- Private keys and service credentials should be stored in hardware-protected environments (HSMs, TPMs, or secure enclaves).

D.1.1.4 Authorisation

D.1.1.4.1 Administrative Authorisation

Implementation Notes:

- Basic (REQ-H-ADMIN-AUTHZ-001)
 - Default-deny can be achieved by ensuring that new accounts have no privileges until explicitly assigned.
 - Administrative interfaces (CLI, API, web console) should reject all actions unless an authenticated account has been authorized.
 - Example: disabling “anonymous” or “guest” administrative access by default.
- Elevated (REQ-H-ADMIN-AUTHZ-002)
 - RBAC: Roles such as “viewer,” “operator,” “administrator” with strictly scoped permissions.
 - ABAC: Access decisions based on attributes such as time of day, source network, or device trust level.
 - Implementation may rely on built-in hypervisor mechanisms (e.g., vSphere roles, libvirt polkit rules) or external identity providers.
- Advanced (REQ-H-ADMIN-AUTHZ-003)
 - Separation of Duties (SoD): Different administrators required for provisioning vs. approval.
 - Just-in-Time (JIT) elevation: Temporary elevation of privileges after MFA approval, automatically revoked after use.
 - Integration with Policy Decision Points (PDPs) or Zero Trust identity systems (e.g., XACML-based PDPs, OPA/Gatekeeper).

D.1.1.4.2 Service Authorisation

Implementation Notes:

- Basic (REQ-H-SERV-AUTHZ-001)
 - Default-deny for service accounts can be enforced by requiring explicit role assignment for each service.

- Example: a monitoring service account is only granted “read telemetry” rights, not “modify configuration.”
 - Automatic lockout after failed attempts prevents brute-force escalation.
- Elevated (REQ-H-SERV-AUTHZ-002)
 - Service accounts should be bound to specific API scopes or resource permissions.
 - Example: Backup service account authorized only for VM snapshot operations, not for VM lifecycle actions.
 - Enforcement can be implemented via signed API tokens mapped to least-privilege roles.
- Advanced (REQ-H-SERV-AUTHZ-003)
 - Binding permissions to cryptographic identities (e.g., X.509 service certificates) ensures strong service-to-service trust.
 - Context-aware enforcement may use runtime signals (e.g., workload identity, network segment, attested state of the service).
 - Integration with external authorization frameworks (e.g., SPIFFE/SPIRE, OAuth2-based PDP, Zero Trust service mesh like Istio).

D.1.1.5 Confidentiality Protection

D.1.1.6 Availability and Resilience

Implementation Notes

Type I

- Basic: Use built-in watchdog timers, fault detection, and service restart mechanisms (e.g., restart management daemon without reboot).
- Elevated: Implement hypervisor clustering with shared storage for VM failover (e.g., VMware vSphere HA, XenServer High Availability).
- Advanced: Continuous memory/state replication to standby hosts for seamless failover (e.g., VMware Fault Tolerance, Xen live state replication).

Type II

- Basic: Rely on host OS process supervision (systemd, init scripts) to restart failed hypervisor processes.
- Elevated: Integrate with host OS clustering frameworks (e.g., Pacemaker, Corosync) to enable VM failover between hosts.
- Advanced: Use host-level live migration with continuous replication (e.g., KVM with QEMU live replication, Hyper-V Replica).

D.1.1.7 Logging

Implementation Notes

Type I

- Basic: Log admin actions to local log files (e.g., /var/log/hypervisor.log) with access restricted to privileged accounts.
- Elevated: Forward logs to remote syslog or SIEM (e.g., syslog-ng, Splunk forwarder) over TLS.
- Advanced: Apply cryptographic signing (e.g., RSA/ECDSA) and trusted timestamping (RFC 3161) to log records; store integrity proofs in TPM.

Type II

- Basic: Use host OS logging facilities (syslog, journald) with permissions restricting VM access.
- Elevated: Configure log forwarding from host OS to secure logging infrastructure with mutual TLS authentication.
- Advanced: Sign and timestamp logs at the host OS level; verify integrity during incident response.

D.1.1.8 Patches and Updates

Implementation Notes

Type I

- Basic: Vendor-supplied update mechanism to apply security patches (e.g., ESXi Offline Bundles).

- Elevated: Digital signature validation of patch files before installation (e.g., RSA-signed VIB packages in ESXi).
- Advanced: Support for live kernel/hypervisor patching (e.g., VMware ESXi Quick Boot, Xen live patching) without downtime.

Type II

- Basic: Apply patches through host OS package management (e.g., `apt`, `yum`) without reinstalling hypervisor modules.
- Elevated: Enable signed package verification via GPG keys managed by vendor or administrator.
- Advanced: Live patching of host kernel modules (e.g., `Ksplice`, `kpatch`) to avoid VM downtime.

D.1.1.9 Secure Configuration

Implementation Notes

Type I

- Basic: Disable unused services and features in hypervisor configuration (e.g., disable unneeded vSwitches, unused VM templates).
- Elevated: Implement configuration validation tools (e.g., VMware Host Profile compliance check) to reject insecure settings.
- Advanced: Auto-apply secure baselines (e.g., DISA STIG, CIS Benchmarks) at install or on schedule.

Type II

- Basic: Disable unused host OS services that could be exploited by guests (e.g., SSH, HTTP daemons not needed for hypervisor).
- Elevated: Validate kernel and module parameters against secure defaults before applying changes.
- Advanced: Integrate with host OS compliance frameworks (e.g., OpenSCAP) to enforce secure baselines automatically.

D.1.1.10 Secure by Default

Implementation Notes

Type I

- Basic: Ship with minimal service set enabled, default firewall rules blocking inbound management access except from trusted networks.
- Elevated: Require explicit admin action to enable insecure features (e.g., promiscuous mode, unencrypted vMotion).
- Advanced: Detect sensitive workloads at deployment (e.g., financial or classified) and auto-enable stronger protections (e.g., VM memory encryption, stricter CPU scheduling isolation).

Type II

- Basic: Ship with minimal host OS services enabled, restrictive default SELinux/AppArmor profiles.
- Elevated: Require explicit enabling of high-risk kernel parameters (e.g., KVM nested virtualization) with security warnings.
- Advanced: Automatically harden kernel/network parameters based on environment (e.g., apply stricter `sysctl` settings for public cloud deployment).

D.1.1.11 Data Minimisation

Implementation Notes

Type I

- Basic: Configure hypervisor logging and telemetry modules to exclude sensitive fields such as passwords, cryptographic keys, and complete VM memory dumps; limit diagnostic exports to operationally necessary data only.
- Elevated: Provide administrator-accessible settings to disable specific log categories, reduce telemetry granularity, and define retention policies (e.g., automatic deletion after 30 days).
- Advanced: Implement automatic redaction/masking of sensitive values in logs, truncate oversized payloads in diagnostics, and delete temporary debug files automatically after export.

Type II

- Basic: Configure host OS logging, monitoring, and diagnostic tools to exclude sensitive information from outputs accessible to the hypervisor or guests; disable unnecessary debug logging in production builds.
- Elevated: Offer configuration parameters in the host OS or hypervisor integration layer to control log categories, telemetry sampling rates, and retention periods for stored logs.
- Advanced: Apply automated filtering in the host OS logging pipeline to redact sensitive fields before forwarding to the hypervisor or external collectors and enable automatic cleanup of transient diagnostic data.

D.1.1.12 Time Synchronisation

Implementation Notes

- Basic: NTP.
- Elevated: NTP or PTP (Precision Time Protocol) with redundancy.
- Advanced: NTS for NTP or IEEE 1588 PTP with security extensions

D.1.2 Management and Orchestration System

D.2 CES

D.2.1 CRS

D.2.1.1 Isolation

D.2.1.2 Integrity Protection

D.2.1.3 Authentication

D.2.1.4 Authorisation

D.2.1.5 Confidentiality Protection

D.2.1.6 Availability and Resilience

D.2.1.7 Logging

D.2.1.8 Patches and Updates

D.2.1.9 Secure Configuration

D.2.1.10 Secure by Default

D.2.1.11 Data Minimisation

D.2.2 CE

D.2.3 CO

Annex (informative): Bibliography

Annex (informative): Change history

Date	Version	Information about changes
June 2025	0.0.1	- Added representative use cases for hypervisors and container runtime systems, classified by risk level. - Defined cumulative security levels (Basic, Standard, Advanced) to guide requirement selection. - Defined threats - Defined generic security requirements - Defined instantiated security requirements for isolation, integrity, authentication, authorisation and confidentiality with assigned security levels across product types (Type 1/2 Hypervisors, CRS). - Added Annex A: Risk assessment methodology - Added Annex B: Mapping between GR requirements and CRA. - Added Annex C: Threat-to-Requirement-to-Control mapping table for traceability. - Added Annex D: Illustrative security controls
August 2025	0.0.2	- Extend the scope to cover the entire container execution stack, including the Container Runtime System (CRS), the Container Engine (CE), and the Container Orchestrator (CO). - Extend the scope to cover the entire virtualisation execution stack, including the hypervisor and the orchestration and management system. - Update scope figures. - Replace "security level" with "security category level". - Add new requirements specific to the container engine (CE) and container orchestrator (CO). - Revise existing CRS requirements to align with the extended scope and updated terminology. - Update the risk assessment methodology in Annex A to: - Clarify how threats are factored into risk determination. - Describe how to assess the overall product risk level. - Introduce an "Assessment" clause, including an example test case for one of the defined security requirements. - Replace the "Assumptions" clause with a new "Operational Environment Objectives" clause.
August 2025	0.0.3	- Updated scope figures. - Revised list of threats, now with specific threats defined per component. - New clause on Objectives for the Operational Environment, replacing the previous <i>Assumptions</i> chapter. - New and updated requirements, including: - New requirements for the management and orchestration system. - Updates to isolation and integrity requirements for hypervisors. - Addition of three options for structuring the assessment of requirements: - Test case-based approach - Simplified PT2 - ETSI TS 103 645 / ETSI EN 303 645 approach
September 2025	0.0.4	- Addressed new comments received (see Excel sheet v5). - Updated the titles of representative use cases in Clause 5. - Updated requirement categorisation and selection approaches in Clause 6: - The label Enhanced has been replaced by Advanced. - A new option for creating a custom security profile has been introduced for cases where the product use case does not fit one of the representative use cases in the standard, or where the manufacturer wishes to conduct a dedicated risk assessment. - A categorisation approach: Cumulative, Exclusive, and Single - has been introduced to clarify the risk-based selection of requirements. - Transformed generic requirements into security objectives (Clause 8). - Updated VES and CES requirements following received comments; additional elements have been added, and the new categorisation approach has been applied to a subset of them (see Clauses 10 and 11). - Added new requirements on vulnerability handling and SBOM in Clause 12. - Updated the risk assessment methodology: simplification of risk factors and application of the updated risk factors to the assessment of use cases. - Moved implementation notes intended as guidance for manufacturers from the requirements clauses to Annex D.

Date	Version	Information about changes
September 2025	0.0.5	• Addressed new comments received from ETSI. • Added a time synchronisation objective and related requirements for the hypervisor, with implementation notes included in Annex D. • Completed and updated authentication requirements for the hypervisor, covering both administrative and service authentication. • Annex A: CRA essential requirements have been mapped to the existing hypervisor requirements. • Annex C: Now designated as a normative annex. Threats to hypervisor requirements are partially mapped as a sample for presentation; this mapping will be completed once a stable set of requirements with fixed IDs is available. • Reordered clauses: The clause defining labels, categories, and SCLs, along with the approaches for applying requirements, has been moved to appear just before the requirements clause, making the document easier to follow. • Updated use case descriptions, including illustrative business applications and revised example environments. • Requirement categorisation, labelling, and classification approaches have been clarified for manufacturers. The guidance is now more direct and less perspective-based. • The label Standard has been replaced by Elevated.
October 2025	0.0.6	• Conformance assessment approach described, including evaluation of hypervisor isolation and boot chain integrity requirements. • Incorporated new comments received. • Added definitions of Tenant and Multi-Tenancy. • Slightly updated use cases UC-V3 and UC-C3 to also cover multi-tenancy scenarios within the same organization. • Updated risk assessment methodology applied to UC-V1/2/3, with risk evaluated per threat for hypervisors. • Mapped hypervisor threats to corresponding technical requirements to CRA requirements. • Moved implementation notes to Annex D. • Replaced the 'Label' term with 'Class'.

History

Document history		
<Version#>	<Month Year>	<Milestone>